Jazyk C

Praktické programování pro pokročilé

Petr Gregor

ICT Pro

4.-5. listopadu 2026

Petr Gregor

- Lektor
  - Python
  - Java
  - C/C++/C#
  - AI
  - Data Science
- Analytik na MUNI
- Programátor (Pascal, C, C++, C#, Java, JavaScript, PHP, VB, Python, Ruby,...)
- 25+ let zkušeností s výukou
- 20+ let zkušeností s programováním



ictPRO

Harmonogram kurzu (3 dny × 8 hodin)

1. Možnosti programovacího jazyka C
2. Datové typy na různých architekturách
3. Práce s bity a binárními daty
4. Zpracovávání chyb
5. Ukazatele do paměti a životnost objektů
6. Pravidla pro aliasing
7. Alokace paměti
8. Hromadná alokace paměti
9. Dynamické datové struktury pro embedded
10. Specifika embedded programování
11. Přerušení (ISR) a sdílená data
12. Úlohy a synchronizace v RTOS
13. Cache, DMA a zarovnání dat
14. Struktury a abstrakce
15. Modularita a návrh testovatelného firmware
16. Psaní testovatelného kódu a mockování
17. Regresní testování a ladění chyb
18. Tvorba korektních programů a diagnostika
19. Polymorfismus v jazyce C
20. Výčtový polymorfismus
21. Rekurzivní volání
22. Možnosti generického programování
23. Objektově orientované programování
24. Bonus: Externí knihovny
25. Finální projekt (Capstone)

Možnosti programovacího jazyka C

Konzervativní rozvoj a standard C23

Evoluce jazyka s důrazem na zpětnou kompatibilitu

- **Filozofie standardizace ISO C:**
  - Hlavním cílem výboru (WG14) je nerozbít existující miliardy řádků kódu.
  - Změny jsou přijímány konzervativně, často jako standardizace již existujících rozšíření kompilátorů (GCC, Clang).
- **Příchod standardu C23 (ISO/IEC 9899:2024):**
  - Odstranění historických archaismů (např. K&R definice funkcí bez typů).
  - Přiblížení k moderním standardům bez zavedení skryté režie (žádné výjimky ani garbage collector).
  - Nativní podpora pro klíčová slova `bool`, `true`, `false`, `alignas`, `alignof` bez nutnosti vkládat hlavičkové soubory.

Novinky ve standardu C23 v praxi

Modernizace typového systému a literálů

```c
// Modern C23 features in practice
#include <stdio.h>
#include <stddef.h>

// constexpr guarantees evaluation at compile-time
constexpr size_t DEFAULT_BUFFER_SIZE = 1024;
// Binary literals and digit separators for better readability
constexpr unsigned int HARDWARE_MASK = 0b1010'0000'1111'0000;

void process_data(const uint8_t *payload, size_t length) {
    // nullptr replaces the legacy NULL macro for pointer types
    if (payload == nullptr) {
        return;
    }

    // typeof allows type deduction without evaluating the expression
    typeof(length) processed_bytes = 0;

    while (processed_bytes < length) {
        // Processing logic here
        processed_bytes++;
    }
}
```

Unikátní postavení jazyka C v praxi

Kde C dominuje a proč je nenahraditelné

- **Základ moderní infrastruktury:**
  - **Jádra OS**: Linux, Windows (NT kernel), macOS (XNU), RTOS (FreeRTOS, Zephyr).
  - **Běhová prostředí (Runtimes)**: Většina virtuálních strojů a interpretů (CPython, JVM, V8 JavaScript) je napsána v C nebo C++.
- **Univerzální systémové rozhraní (C ABI):**
  - C definuje standardní *Application Binary Interface*, kterému rozumí všechny ostatní jazyky.
  - Pokud chce Rust, Python nebo C# komunikovat s operačním systémem nebo jinou knihovnou, dělá to prostřednictvím funkcí exportovaných v C ABI.
- **Minimální běhová režie:**
  - Program v C nepotřebuje před spuštěním `main()` inicializovat složité běhové prostředí, což je klíčové pro mikrokontroléry s jednotkami kilobajtů RAM.

Praktická ukázka: Univerzální C ABI

Návrh rozhraní komunikujícího s libovolným jiným jazykem

```c
// sensor_api.h - Exported C interface (Stable ABI)
// This header can be easily consumed by Python (ctypes) or Rust (FFI)

#include <stdint.h>
#include <stdbool.h>

// Struct with explicitly defined sizes to prevent ABI mismatch across platforms
typedef struct {
    uint64_t timestamp_us;
    float temperature_celsius;
    uint32_t error_flags;
} SensorReading;

// Opaque handle for encapsulating internal state
typedef struct SensorContext SensorContext;

// Public API functions without name mangling (unlike C++)
SensorContext* sensor_create_instance(void);
bool sensor_fetch_data(SensorContext *ctx, SensorReading *out_reading);
void sensor_destroy_instance(SensorContext *ctx);
```

Rozdíly oproti moderním alternativám

Srovnání filozofií správy prostředků

Kritérium	Jazyk C (C23)	Jazyk C++	Jazyk Rust
Správa paměti	Manuální (`malloc`/`free`)	RAII, chytré ukazatele	Borrow checker
Typová kontrola	Slabší (implicitní konverze)	Silná (šablony, typy)	Striktní (přísná analýza)
Skrytá režie	**Žádná**	Konstruktory/Destruktory	Žádná (Zero-cost abstractions)
Objem specifikace	Malý ($\sim$700 stran)	Obrovský ($>$ 2000 stran)	Střední, rychle rostoucí
Filozofie chyb	Návratové kódy	Výjimky (často zakázané v HW)	Datové typy `Result<T, E>`

C vs. zbytek světa

Zatímco C++ a Rust poskytují pokročilé abstrakce a bezpečnostní mechanismy už na úrovni kompilátoru, jazyk C nabízí absolutní a transparentní kontrolu nad každým bajtem a taktem procesoru za cenu plné zodpovědnosti programátora.

Praktická ukázka: Explicitní kontrola v C

Absence skrytých operací pod kapotou

```c
#include <stdint.h>
#include <string.h>

typedef struct {
    uint8_t buffer[1024];
    size_t active_length;
} DataPacket;

void process_and_clear_packet(DataPacket *packet) {
    if (packet == nullptr) return;

    // In C++, passing a struct by value might trigger a hidden copy constructor.
    // In C, everything is explicit. We explicitly pass by pointer.

    // If we want to clear sensitive memory, we must explicitly call memset.
    // There are no automatic destructors running when the struct goes out of scope.
    memset(packet->buffer, 0, sizeof(packet->buffer));
    packet->active_length = 0;
}
```

Samostatný úkol: Bezpečný paketový filtr (C23)

Časová dotace: 15–20 minut

Zadání úkolu:
Navrhněte a implementujte modul pro validaci a bezpečné vymazání komunikačního rámce s využitím novinek standardu **C23** a principů stabilního **C ABI**.

Požadavky na implementaci:

1. Pomocí `constexpr` definujte maximální velikost payloadu (např. 64 bajtů) a pomocí binárního literálu s oddělovači (`0b...`) definujte bitovou masku chybových příznaků.
2. Vytvořte strukturu `TelemetryPacket` s explicitní šířkou typů (`<stdint.h>`), která bude obsahovat ID senzoru, pole pro payload, aktivní délku a chybové příznaky.
3. Implementujte funkci `bool packet_validate_and_process(TelemetryPacket *pkt)`:
   - Ošetřete nulový ukazatel pomocí klíčového slova `nullptr`.
   - Pomocí bitové masky zkontrolujte, zda paket neobsahuje kritickou chybu.
   - Využijte `typeof` pro deklaraci lokální řídicí proměnné cyklu a spočítejte kontrolní součet (prostý součet bajtů).
   - Po zpracování explicitně vymažte payload v paměti pomocí `memset`.

Datové typy na různých architekturách

Velikost typu `int` a paměťové modely

Proč `int` neukládá vždy stejný počet bajtů

- **C standard negarantuje pevnou velikost:**
  - Standard ISO C definuje pouze **minimální** rozsahy. Například typ `int` musí mít velikost alespoň 16 bitů (2 bajty).
  - Na 8bitových (AVR) a 16bitových (MSP430) architekturách má `int` typicky 16 bitů.
  - Na 32bitových (ARM Cortex-M) a 64bitových (x86_64) systémech má `int` většinou 32 bitů.
- **Datové modely (Data Models):**
  - **ILP32**: `int`, `long` i ukazatele mají 32 bitů (běžné 32bitové MCU).
  - **LP64**: `long` a ukazatele mají 64 bitů, ale `int` zůstává 32bitový (např. Linux x86_64).
  - **LLP64**: Pouze `long long` a ukazatele mají 64 bitů (např. Windows x64).

Přehled celočíselných typů napříč architekturami

Porovnání velikosti v bajtech, rozsahu hodnot a potřebných hlaviček

Datový typ	Hlavička	8/16-bit *(AVR)*	32-bit *(ARM CM)*	64-bit *(Linux LP64)*	Rozsah hodnot *(dle šířky typu)*
`char` / `uint8_t`	`<stdint.h>`	1 B	1 B	1 B	0 až 255 (pro `uint8_t`)
`short` / `int16_t`	`<stdint.h>`	2 B	2 B	2 B	−32 768 až 32 767
`uint16_t`	`<stdint.h>`	2 B	2 B	2 B	0 až 65 535
`int`	*vestavěný*	**2 B**	**4 B**	**4 B**	2B: ±32 k | 4B: $\pm 2{,}14 \times 10^{9}$
`unsigned int`	*vestavěný*	**2 B**	**4 B**	**4 B**	2B: 65 k | 4B: $4{,}29 \times 10^{9}$
`long`	*vestavěný*	4 B	**4 B**	**8 B***	4B: $\pm 2 \times 10^{9}$ | 8B: $\pm 9 \times 10^{18}$
`long long`	*vestavěný*	8 B	8 B	8 B	$-9{,}22 \times 10^{18}$ až $+9{,}22 \times 10^{18}$
`int32_t`	`<stdint.h>`	4 B	4 B	4 B	−2 147 483 648 až +2 147 483 647
`uint32_t`	`<stdint.h>`	4 B	4 B	4 B	0 až 4 294 967 295
`uint64_t`	`<stdint.h>`	8 B	8 B	8 B	0 až $1{,}84 \times 10^{19}$ ($2^{64} - 1$)
`uint_fast16_t`	`<stdint.h>`	2 B	**4 B**	**8 B**	Nejrychlejší typ s šířkou $\geq$ 16 bitů
`atomic_uint_fast32_t`	`<stdatomic.h>`	4 B**	4 B	8 B	Garantovaně atomický přístup ($\geq$ 32b)

* Poznámka: Na 64bitových Windows (model LLP64) zůstává `long` 4bajtový.
** Na 8bitových MCU bez atomických instrukcí (AVR) vyžaduje softwarovou emulaci se zákazem přerušení.

Typy s plovoucí řádovou čárkou (IEEE 754)

Velikost, přesnost a hardwarová podpora (FPU) na různých platformách

Datový typ	Hlavička	8-bit AVR *(avr-gcc)*	32-bit *(ARM CM)*	64-bit *(x86_64)*	Rozsah a přesnost *(dle IEEE 754)*
`float`	`<float.h>`	4 B	4 B	4 B	$\pm 3{,}4 \times 10^{38}$ ($\sim$ 7 platných číslic)
`double`	`<float.h>`	**4 B***	8 B	8 B	$\pm 1{,}7 \times 10^{308}$ ($\sim$ 15 platných číslic)
`long double`	`<float.h>`	**4 B***	8 B	**10 / 16 B****	$\pm 1{,}1 \times 10^{4\,932}$ (19 až 33 číslic)
`_Float16`	*C23 / GCC*	–	2 B	2 B	$\pm 6{,}55 \times 10^{4}$ ($\sim$ 3 platné číslice)

Kritický dopad na výkon v Embedded C

- **Cortex-M0/M3 (bez FPU):** Veškeré výpočty s `float` i `double` jsou emulovány softwarově (stovky taktů CPU).
- **Cortex-M4F / M33 (Single-Precision FPU):** Hardwarově akceleruje pouze 4B `float` (1 takt). Zapomenutý sufix `f` u literálu (např. `x * 3.14` místo `x * 3.14f`) vynutí pomalý softwarový převod na 8B `double`!
- **Cortex-M7 (Double-Precision FPU):** Volitelně akceleruje hardwarově jak 4B `float`, tak 8B `double`.

* V překladači `avr-gcc` je `double` z výkonnostních důvodů standardně zkrácen na 4 bajty (shodný s `float`).
** Na x86_64 Linuxu zabírá `long double` 16 bajtů v paměti (80bitový x87 formát s paddingem), na Windows (MSVC) 8 bajtů.

Adresové, indexové a speciální typy

Datové typy přizpůsobující svou šířku adresové sběrnici procesoru

Datový typ	Hlavička	16-bit *(MSP430)*	32-bit *(ARM CM)*	64-bit *(x86_64)*	Účel a rozsah hodnot
`void*` (ukazatel)	*vestavěný*	2 B	4 B	8 B	Paměťová adresa (2^{16}, 2^{32} nebo 2^{64} B)
`size_t`	`<stddef.h>`	2 B	4 B	8 B	Bezznaménková velikost objektu (`sizeof`)
`ptrdiff_t`	`<stddef.h>`	2 B	4 B	8 B	Znaménkový rozdíl dvou ukazatelů ($p_1 - p_2$)
`uintptr_t`	`<stdint.h>`	2 B	4 B	8 B	Celé číslo bezpečně pojmující ukazatel
`intmax_t`	`<stdint.h>`	8 B	8 B	8 B	Největší podporovaný celočíselný typ
`bool`	*C23 vestavěný**	1 B	1 B	1 B	Logická hodnota: pouze `false` (0) / `true` (1)
`char` (znaky)	*vestavěný*	1 B	1 B	1 B	**Pozor:** Na x86 je *signed*, na ARM je *unsigned*!

Zlaté pravidlo přenositelnosti mezi ARM a x86

Standard C nespecifikuje, zda je samotný typ `char` znaménkový nebo bezznaménkový! Na architektuře **x86_64** má `char` rozsah −128 až 127, zatímco na **ARM Cortex-M** má rozsah 0 až 255. Pro číselné bajty proto vždy používejte `uint8_t` nebo `int8_t`.

* **Před standardem C23 (v C99/C11) vyžadoval typ `bool` vložení hlavičkového souboru `<stdbool.h>`.**

Praktická ukázka: Bezpečné datové typy

Použití standardní knihovny <stdint.h>

```c
#include <stdio.h>
#include <stdint.h>

void process_sensor_data(void) {
    // DANGEROUS in embedded: Size depends on architecture!
    // Could overflow if 'int' is 16-bit (max positive value 32767).
    int sensor_reading = 45000;

    // SAFE: Strictly 32-bit unsigned integer regardless of MCU/CPU.
    uint32_t precise_reading = 45000;

    // Pointer sizes also vary (16, 32, or 64 bits).
    // uintptr_t guarantees an integer type large enough to hold a pointer safely.
    uintptr_t memory_address = (uintptr_t)&precise_reading;

    // Type casting to size_t for safe printf formatting
    printf("Reading: %u at 0x%zx\n", precise_reading, (size_t)memory_address);
}
```

Samostatný úkol: Přenositelný dekodér telemetrie

Časová dotace: 15–20 minut

Zadání úkolu:
Původní 16bitový firmware meteostanice (AVR) byl portován na 64bitové PC, ale výpočty po přenosu selhávají kvůli změně šířky typu `int` a velikosti ukazatelů.

Požadavky na implementaci:

1. Definujte strukturu `RawAdcFrame` využívající striktně typy s pevnou šířkou z `<stdint.h>`:
   - 16bitové znaménkové číslo pro teplotu v desetinách °C (`int16_t`),
   - 32bitové bezznaménkové číslo pro atmosférický tlak v Pascalech (`uint32_t`),
   - 8bitové bezznaménkové číslo pro stav baterie v procentech (`uint8_t`).
2. Implementujte funkci `bool compute_altitude_factor(const RawAdcFrame *frame, int32_t *out_factor)`, která vynásobí teplotu a tlak bez rizika přetečení na 16bitovém MCU (použijte mezivýpočet v `int64_t`).
3. Vypište na konzoli velikosti jednotlivých položek pomocí `sizeof` a paměťovou adresu struktury bezpečně uloženou do typu `uintptr_t`.

Práce s bity a binárními daty

Bitové položky (Bit-fields)

Úspora paměti a mapování registrů

- **Princip bitových položek:**
  - Umožňují deklarovat členy struktury s přesně definovanou šířkou v bitech pomocí syntaxe `: pocet_bitu`.
  - Ideální pro maximální úsporu paměti a pro přímé mapování stavových hardwarových registrů.
- **Nedefinované chování a omezení:**
  - **Pořadí bitů a zarovnání**: To, zda kompilátor skládá bity zleva doprava nebo zprava doleva, je nedefinované chování závislé čistě na překladači a architektuře.
  - Na bitovou položku nelze aplikovat operátor adresy (`&`) a nelze použít standardní ukazatel, protože adresa v C odkazuje nejméně na celé bajty.

Praktická ukázka: Bitové položky

Efektivní zapouzdření stavu systému v jedné struktuře

```c
#include <stdint.h>
#include <stdio.h>

// Packing multiple flags and smaller values into a single byte footprint
typedef struct {
    uint8_t system_ready  : 1; // 1 bit (0 or 1)
    uint8_t error_active  : 1; // 1 bit
    uint8_t mode_selector : 3; // 3 bits (values 0 to 7)
    uint8_t sensor_status : 2; // 2 bits (values 0 to 3)
    uint8_t reserved      : 1; // 1 bit padding to fill exactly 8 bits
} ControllerStatus;

void check_system(ControllerStatus status) {
    if (status.system_ready) {
        printf("System is ready. Current Mode: %u\n", status.mode_selector);
    }
}
```

Endianness (Uspořádání bajtů)

Little-endian vs. Big-endian

- **Little-endian vs. Big-endian:**
  - **Little-endian (LE)**: Nejméně významný bajt (LSB) se ukládá na nejnižší paměťovou adresu (např. ARM Cortex, x86).
  - **Big-endian (BE)**: Nejvýznamnější bajt (MSB) se ukládá na nejnižší adresu (běžné pro síťový přenos - *network byte order*).
- **Konverze a převody:**
  - Při komunikaci mezi odlišnými architekturami (nebo odesílání po síti) musí dojít k převodu uspořádání bajtů, jinak dojde k dezinterpretaci hodnot.
  - Realizuje se buď manuálně (bitové posuny a masky), nebo pomocí optimalizovaných vestavěných funkcí překladače (např. `__builtin_bswap32`).

Praktická ukázka: Detekce a konverze endianity

Ukázka runtime detekce a manuálního otočení bajtů

```c
#include <stdint.h>
#include <stdbool.h>

// Checks host endianness at runtime
bool is_little_endian(void) {
    uint16_t test_value = 0x0001;
    // Reading the first byte: if it contains 0x01, it is Little-Endian
    return (*(uint8_t *)&test_value == 1);
}

// Manually swaps bytes of a 32-bit unsigned integer using shifts and masks
uint32_t swap_uint32(uint32_t val) {
    return ((val & 0x000000FFU) << 24) |
           ((val & 0x0000FF00U) << 8)  |
           ((val & 0x00FF0000U) >> 8)  |
           ((val & 0xFF000000U) >> 24);
}
```

Serializace struktur

Problém s kopírováním paměti a bezpečný přenos

- **Problém s přímým kopírováním struktur:**
  - Odeslání paměťového obrazu struktury je nebezpečné kvůli výplňovým bajtům (padding) kompilátoru a neřeší problém s endianitou.
- **Atribut packed (`__attribute__((packed))`):**
  - Příkaz pro překladač, aby odstranil veškerý vnitřní padding struktury.
  - *Nevýhoda:* Může způsobit nezarovnaný přístup do paměti (*Unaligned Access*), což na některých procesorech (např. starší ARM) vyvolá pád (HardFault). Navíc to stále neřeší endianitu.
- **Bezpečné byte-by-byte parsování / serializace:**
  - Manuální naplnění pole bajtů (`uint8_t` bufferu) položku po položce pomocí bitových posunů.
  - Zcela imunní vůči chování kompilátoru, paddingu i endianitě.

Praktická ukázka: Bezpečná serializace

Explicitní ukládání dat do bufferu bez ohledu na padding

```c
#include <stdint.h>
#include <stddef.h>

typedef struct {
    uint16_t sensor_id;
    uint32_t timestamp;
} PayloadData;

// Safely serializes struct into a byte stream (Big-Endian network order)
void serialize_payload(const PayloadData *data, uint8_t *buffer) {
    if (data == nullptr || buffer == nullptr) return;

    // Pack sensor_id (16-bit)
    buffer[0] = (uint8_t)(data->sensor_id >> 8);
    buffer[1] = (uint8_t)(data->sensor_id & 0xFF);

    // Pack timestamp (32-bit)
    buffer[2] = (uint8_t)(data->timestamp >> 24);
    buffer[3] = (uint8_t)(data->timestamp >> 16);
    buffer[4] = (uint8_t)(data->timestamp >> 8);
    buffer[5] = (uint8_t)(data->timestamp & 0xFF);
}
```

Samostatný úkol: Serializace a deserializace CAN rámce

Časová dotace: 15–20 minut

Zadání úkolu:
Navrhněte konverzní modul pro přenos řídicího rámce motoru po sběrnici ve formátu **Big-Endian** (síťové pořadí bajtů) bez spolehnutí na kompilátorem vkládaný padding.

Požadavky na implementaci:

1. Definujte 1bajtovou strukturu bitových položek (*bit-fields*) `MotorFlags` obsahující: `enable` (1 bit), `direction` (1 bit), `error_code` (4 bity) a `reserved` (2 bity).
2. Definujte strukturu `MotorCommand` obsahující: `uint16_t rpm`, `uint32_t timestamp_ms` a `uint8_t status_byte` (sbalené příznaky).
3. Implementujte funkce `serialize_motor_cmd` a `deserialize_motor_cmd`, které převedou strukturu do pole `uint8_t buffer[7]` (a zpět) ve formátu **Big-Endian** pomocí bitových posunů a masek.



Zpracovávání chyb

Ošetřování návratových hodnot

Tradiční přístupy k detekci chyb v jazyce C

- **Absence výjimek (Exceptions):**
  - Jazyk C nemá bloky `try`, `catch` ani automatické probublávání chyb.
  - Veškeré chyby musí být předávány explicitně v toku programu.

- **Nejčastější idiomy pro předávání chyb:**
  - **Návratové kódy (*Return Codes*)**: Funkce vrací celočíselný status (např. `0` pro úspěch, `<0` pro chybu).
  - **In-band signalizace**: Zvláštní hodnota v platném rozsahu (např. `nullptr` nebo `-1`) indikuje selhání.
  - **Out-parametry**: Funkce vrací status, samotná data jsou zapisována do paměti poskytnuté volajícím přes ukazatel.
  - **Globální `errno`**: Tradiční (ale v multithreadingu problematický) přístup POSIX knihoven.

Praktická ukázka: Návratové kódy

Definice chybových stavů pomocí výčtového typu

```c
#include <stdint.h>
#include <stdbool.h>

// Explicitly defined error codes using enum
typedef enum {
    STATUS_OK = 0,
    STATUS_ERR_INVALID_PARAM = -1,
    STATUS_ERR_TIMEOUT = -2,
    STATUS_ERR_HARDWARE = -3
} ErrorStatus;

// Data is returned via pointer, status is the actual return value
ErrorStatus read_sensor_data(uint8_t sensor_id, uint32_t *out_data) {
    if (sensor_id > 10 || out_data == nullptr) {
        return STATUS_ERR_INVALID_PARAM; // Early return on invalid input
    }

    // Simulate hardware read success
    *out_data = 4096;
    return STATUS_OK;
}
```

Atribut [[nodiscard]] (C23)

Vynucení kontroly chyb na úrovni kompilátoru

- **Problém ignorovaných chyb:**
  - Programátor může zavolat funkci a zapomenout zpracovat její návratový kód.
  - To vede k tichému selhání (tzv. *silent failure*), kdy program pokračuje s neplatnými daty nebo v neplatném stavu.
- **Řešení ve standardu C23:**
  - Zavedení standardního atributu `[[nodiscard]]` (inspirováno jazykem C++17).
  - Zapisuje se přímo do deklarace funkce před návratový typ.
  - Pokud volající ignoruje návratovou hodnotu takové funkce, překladač vygeneruje **varování (warning)**.
  - V kombinaci s přepínačem `-Werror` to znamená přerušení kompilace.

Praktická ukázka: Použití atributu

Ochrana kritických funkcí před ignorováním výsledku

```c
#include <stdint.h>

typedef enum { SYS_OK, SYS_FAIL } SystemStatus;

// C23: Compiler forces the caller to check the returned SystemStatus
[[nodiscard]] SystemStatus system_initialize(void) {
    // Perform critical hardware setup
    return SYS_OK;
}

void application_start(void) {
    // WRONG: Ignoring the return value!
    // Compiler warning: ignoring return value of function declared with 'nodiscard'
    system_initialize();

    // CORRECT: Explicitly handling the return value
    SystemStatus status = system_initialize();
    if (status != SYS_OK) {
        // Handle critical failure
        while (1) {}
    }
}
```

Unwinding selhání pomocí goto

Náhrada za RAII a bezpečný clean-up pattern

- **Proč je goto užitečné:**
  - Ačkoliv je `goto` často (a právem) kritizováno za vytváření nepřehledného *spaghetti kódu*, pro ošetřování chyb v C je to standardní, uznávaný vzor (používaný masivně např. v jádře Linuxu).
- **Single-Exit Pattern (Odvíjení):**
  - Funkce, která alokuje více prostředků (paměť, hardwarové handlery, mutexy), potřebuje v případě chyby vše korektně uvolnit.
  - Bez `goto` vzniká nepřehledná pyramida vnořených `if` bloků (*Arrow anti-pattern*) nebo hrozí duplikace úklidového kódu.
  - S `goto` je úklid řešen na konci funkce bloky štítků seřazenými v opačném pořadí, než probíhala alokace (*Reverse-order unwinding*).

Praktická ukázka: Clean-up pattern s goto

Linearizace kódu a prevence úniku prostředků

```c
#include <stdbool.h>
#include <stdlib.h>

[[nodiscard]] bool process_stream(void) {
    bool success = false;
    void *buffer = malloc(1024);
    if (!buffer) goto cleanup_none; // Jump to the end

    void *hardware_lock = acquire_hardware();
    if (!hardware_lock) goto cleanup_buffer; // Undo step 1

    if (!perform_data_transfer(buffer, hardware_lock)) {
        goto cleanup_all; // Undo step 1 and 2
    }

    success = true; // Reached only if everything succeeds

// Unwinding blocks in reverse order of acquisition
cleanup_all:
    release_hardware(hardware_lock);
cleanup_buffer:
    free(buffer);
cleanup_none:
    return success;
```

Samostatný úkol: Transakční inicializace SPI flash

Časová dotace: 15–20 minut

Zadání úkolu:
Implementujte bezpečnou inicializační sekvenci pro externí SPI Flash paměť, která alokuje více systémových prostředků a v případě selhání libovolného kroku provede korektní odvinutí (*unwinding*) pomocí `goto`.

Požadavky na implementaci:

1. Definujte výčtový typ návratových kódů `FlashStatus` (`FLASH_OK`, `FLASH_ERR_ALLOC`, `FLASH_ERR_BUS_LOCK`, `FLASH_ERR_VERIFY`).
2. Opatřete hlavní funkci `flash_storage_mount(bool simulate_verify_fail)` atributem `[[nodiscard]]`.
3. Funkce postupně: (a) alokuje vyrovnávací buffer (256 B), (b) uzamkne SPI sběrnici, (c) aktivuje Chip Select (CS) pin a (d) ověří ID čipu.
4. Pokud kterýkoliv krok selže, použijte **Single-Exit Pattern s** `goto` k uvolnění pouze těch prostředků, které již byly získány, v přesně opačném pořadí.

Ukazatele do paměti a životnost objektů

Vlastnictví paměti (Ownership)

Disciplína správy paměti bez automatického Garbage Collectoru

- Jazyk C nemá zabudovaný *borrow checker* (jako Rust) ani destruktory RAII (jako C++).
- **Pravidlo vlastníka (Owner):**
  - Právě jeden subsystém, modul nebo objekt je explicitně odpovědný za alokaci, inicializaci a finální dealokaci paměti.
  - Vlastník rozhoduje o tom, kdy platnost paměti končí.
- **Převod vlastnictví (*Move / Transfer of Ownership*):**
  - Odesilatel předává právo i povinnost uvolnit paměť příjemci (např. vložení zprávy do RTOS fronty).
  - Původní ukazatel musí být okamžitě nastaven na `nullptr`, aby se zamezilo *double-free* a *use-after-free*.

Pravidla pro ukazatele a Pointer Provenance

Ukazatel v C není pouhé číslo

- **Co je to Provenance (původ ukazatele):**
  - Moderní standardy C a optimalizátory nesledují pouze číselnou adresu, ale také to, **ze kterého alokovaného objektu ukazatel vzešel**.
  - Dva ukazatele se stejnou číselnou hodnotou adresy nemusí být zaměnitelné, pokud pocházejí z různých objektů.

- **Pravidla relačních operátorů (<, <=, >, >=):**
  - Relační porovnání dvou ukazatelů je definováno **pouze tehdy**, pokud oba ukazují do téhož pole nebo do stejné struktury.
  - Porovnání ukazatelů ukazujících do dvou nezávislých polí je **Undefined Behavior**!

Praktická ukázka Provenance a limitů aritmetiky

Kompilátor předpokládá nemožnost přetečení mezi poli

```c
#include <stdint.h>
#include <stdbool.h>

static uint8_t rx_buffer[64];
static uint8_t tx_buffer[64];

bool is_pointer_within_rx(const uint8_t *ptr) {
    // VALID: Both pointers derive from the same underlying array object
    return (ptr >= &rx_buffer[0]) && (ptr < &rx_buffer[64]);
}

bool compare_distinct_arrays(const uint8_t *p1, const uint8_t *p2) {
    // UNDEFINED BEHAVIOR if p1 is in rx_buffer and p2 is in tx_buffer!
    // Compiler optimizer may assume this comparison is never reachable
    // and optimize out critical security checks.
    // return (p1 < p2);

    // SAFE WAY: Cast to uintptr_t when comparing raw memory hardware addresses
    return ((uintptr_t)p1 < (uintptr_t)p2);
}
```

Propůjčení paměti (Borrowing)

Bezpečné předávání referencí dalším modulům

- **Princip propůjčení:**
  - Dočasný přístup k datům bez práva a povinnosti uvolnit paměť.
  - Výpůjčka (reference) nesmí **nikdy** přežít samotného vlastníka objektu.
- **Immutable Borrow (Nemĕnná výpůjčka):**
  - Přístup pouze pro čtení chráněný pomocí klíčového slova `const`.
  - Volaná funkce garantuje, že obsah neupraví a nepředá jej funkci, která by ho mohla upravit.
- **Mutable Borrow (Změnitelná výpůjčka):**
  - Běžný ukazatel. Volaná funkce smí modifikovat data, ale nesmí je uvolnit (`free`) ani převzít do vlastnictví.

Propůjčování v signaturách rozhraní

Konvence pro bezpečné rozlišení operací

```
// 1. Immutable Borrow: Read-only access, callee cannot mutate or free
void telemetry_log_packet(const Packet *packet);

// 2. Mutable Borrow: Callee modifies buffer, but caller retains ownership
void packet_compute_crc(Packet *packet);

// 3. Ownership Transfer (Take): Callee takes full ownership and must free
void packet_queue_push(PacketQueue *queue, Packet *packet);

// 4. Safe Invalidation Pattern: Pointer-to-pointer allows resetting caller's handle
void packet_destroy(Packet **packet_ptr) {
    if (packet_ptr != nullptr && *packet_ptr != nullptr) {
        // Free payload or return to pool
        memory_pool_free(*packet_ptr);

        // Invalidate caller pointer immediately to prevent dangling access
        *packet_ptr = nullptr;
    }
}
```

Životnost objektů (Lifetime)

Kategorie paměti a jejich trvání v C

- Jazyk C definuje čtyři základní modely doby platnosti (*storage duration*):
  - **Static**: Platná po celou dobu běhu programu (sekce `.data`, `.bss`, globální a lokální `static` proměnné).
  - **Automatic**: Vázaná na zásobníkový rámec (*stack frame*) funkce nebo lokálního bloku. Zaniká opuštěním bloku!
  - **Allocated**: Dynamicky řízená programátorem (halda přes `malloc`, paměťové bazény, arény).
  - **Thread-local**: Specifická pro vlákno (C11 `_Thread_local`), v malých RTOS často nepodporovaná.
- **Embedded realita:**
  - Zásobník je striktně omezený (často jednotky KiB pro celou aplikaci nebo RTOS task).
  - Přístup k objektu po skončení jeho doby platnosti je **Undefined Behavior (UB)**.

Kritická chyba: Zaniklá platnost (Dangling Pointer)

Předání lokální adresy do asynchronní operace

```c
// DANGEROUS PATTERN: Returning or passing automatic memory asynchronously
#include <stdint.h>
#include <stdbool.h>

void uart_send_dma_start(const uint8_t *buffer, size_t length);

void process_sensor_event(void) {
    // Allocated on the stack (automatic storage duration)
    uint8_t payload[16] = {0xAA, 0x01, 0x02};

    // Hardware DMA transfer is initiated asynchronously in background
    uart_send_dma_start(payload, sizeof(payload));

    // FUNCTION RETURNS IMMEDIATELY: 'payload' stack memory is invalidated!
    // When DMA actually reads payload later, stack is corrupted or overwritten.
}
```

Důsledek v embedded praxi

Zásobníkový rámec funkce `process_sensor_event` je po návratu okamžitě přepsán. DMA pak odesílá náhodná data a způsobuje záhadné chyby.

Samostatný úkol: Bezpečná správa DMA bufferu

Časová dotace: 15–20 minut

Zadání úkolu:
Opravte kritickou chybu zaniklé platnosti (*dangling pointer*) při asynchronním odesílání dat a implementujte jasná pravidla vlastnictví (*ownership* a *borrowing*).

Požadavky na implementaci:

1. Implementujte funkci pro neměnnou výpůjčku (*Immutable Borrow*) `uint8_t compute_crc8(const uint8_t *data, size_t len)`, která pouze čte data bez převzetí vlastnictví.
2. Implementujte funkci `bool is_ptr_in_dma_pool(const uint8_t *ptr)`, která pomocí přetypování na `uintptr_t` bezpečně ověří (bez porušení *Pointer Provenance*), zda daný ukazatel leží uvnitř statického pole `dma_tx_pool`.
3. Implementujte bezpečný deallokační vzor `void release_tx_slot(uint8_t **slot_ptr)`, který po uvolnění okamžitě nastaví volajícímu ukazatel na `nullptr`.

Pravidla pro aliasing

Strict Aliasing Rule

Pravidlo o neshodných typech v ISO C

Definice Strict Aliasing Rule (ISO/IEC 9899)

Dva ukazatele různých typů nesmí ukazovat na stejný paměťový objekt. Překladač předpokládá, že zápis přes typ T_1 nemění objekt typu T_2.

- **Zákonné výjimky ze Strict Aliasingu:**
  - Znakové typy: `char*`, `unsigned char*`, `signed char*` smí aliasovat libovolný jiný typ (pro inspekci bajtů paměti).
  - Znaménkové a bezznaménkové varianty téhož typu (např. `int32_t*` a `uint32_t*`).
  - Položky v rámci stejné `union`.
- **Důsledek porušení:**
  - Přetypování nekompatibilních ukazatelů je **Undefined Behavior**.
  - Při zapnuté optimalizaci (`-O2`, `-O3`) to vede k vynechání kódu nebo k načítání starých (neplatných) hodnot z registrů namísto z paměti.

Type Punning: Chybný vs. korektní přístup

Bezpečná reinterpretace bitů floatu na celočíselný typ

```c
#include <stdint.h>
#include <string.h>

// UNDEFINED BEHAVIOR: Violates strict aliasing rule!
uint32_t float_to_raw_bits_bad(float val) {
    return *(uint32_t *)&val; // DANGEROUS: Compiler may reorder or drop load
}

// SAFE APPROACH 1: Using memcpy (Standard ISO C idiom)
// Modern compilers compile this into a single CPU register move (zero overhead)
uint32_t float_to_raw_bits_safe_memcpy(float val) {
    uint32_t raw_bits;
    memcpy(&raw_bits, &val, sizeof(raw_bits));
    return raw_bits;
}

// SAFE APPROACH 2: Using union (Explicitly defined in C99/C11/C23)
uint32_t float_to_raw_bits_safe_union(float val) {
    union {
        float f;
        uint32_t u;
    } converter;
    converter.f = val;
    return converter.u;
```

Klíčové slovo `restrict`

Slib programátora překladači o výhradním přístupu

- Zavedeno ve standardu C99 pro kvalifikaci ukazatelů.
- **Sémantika a závazek:**
  - Programátor prostřednictvím `restrict` garantuje, že po celou dobu platnosti ukazatele nebude odkazovaný paměťový objekt modifikován přes žádný jiný ukazatel, který je v daném kontextu dostupný.
- **Typický příklad z libc:**
  - `memcpy(void * restrict dest, const void * restrict src, size_t n);`
    Paměťové bloky `dest` a `src` se **nesmí** překrývat. Zápis je nejrychlejší.
  - `memmove(void *dest, const void *src, size_t n);`
    Bez `restrict`. Bloky se překrývat smějí; funkce proto musí bezpečně detekovat směr kopírování, což je pomalejší.

Jak aliasing limituje optimalizátor

Proč překladač nemůže předpokládat nezávislost

- **Neviditelný nepřítel výkonu (Reload from memory):**
  - Pokud překladač nedokáže s jistotou určit, zda se dva ukazatele na stejný typ nepřekrývají, musí se po každém zápisu znovu dotazovat operační paměti (SRAM).
  - Nemůže danou hodnotu po celou dobu výpočtu bezpečně uzamknout do super-rychlého registru CPU.
- **Přínos explicitního slibu:**
  - Opatření kódu správnými aliasovacími mantinely umožňuje plné využití vektorizace (SIMD/DSP instrukce na Cortex-M4/M7).
  - Ve vnitřních cyklech filtrů (FIR/IIR) nebo řídicích algoritmů to může znamenat řádové zrychlení.

Praktický dopad `restrict` na generovaný kód

Eliminace zbytečných přístupů do paměti

```c
#include <stdint.h>
#include <stddef.h>

// WITHOUT restrict: Compiler must reload *factor from RAM every iteration,
// because it assumes writing to output[i] MIGHT accidentally overwrite *factor!
void apply_gain(int32_t *output, const int32_t *input, const int32_t *factor, size_t len) {
    for (size_t i = 0; i < len; ++i) {
        output[i] = input[i] * (*factor);
    }
}

// WITH restrict: Compiler proves factor cannot be mutated by writes to output.
// *factor is loaded ONCE into a CPU register before the loop executes.
void apply_gain_optimized(int32_t * restrict output,
                          const int32_t * restrict input,
                          const int32_t * restrict factor,
                          size_t len) {
    for (size_t i = 0; i < len; ++i) {
        output[i] = input[i] * (*factor);
    }
}
```

Samostatný úkol: Bezpečná konverze floatu a DSP filtr

Časová dotace: 15–20 minut

Zadání úkolu:
Opravte telemetrický modul, který porušuje *Strict Aliasing Rule* při kontrole NaN/Infinity hodnot, a zoptimalizujte výpočetní smyčku digitálního filtru pomocí klíčového slova `restrict`.

Požadavky na implementaci:

1. Implementujte funkci `uint32_t float_extract_exponent(float val)`, která pomocí `memcpy` (nebo `union`) bezpečně přečte bitovou reprezentaci 32bitového čísla `float` (IEEE 754) a pomocí bitového posunu a masky vrátí 8bitový exponent (bity 23 až 30) bez porušení *Strict Aliasingu*.
2. Implementujte funkci `void dsp_scale_and_offset(...)`, která přijímá vstupní pole `in`, výstupní pole `out` a ukazatele na konstanty `scale` a `offset`.
3. Opatřete všechny ukazatele kvalifikátorem `restrict`, aby kompilátor nemusel v každé iteraci znovu načítat `*scale` a `*offset` z paměti RAM.

Alokace paměti

Organizace paměti mikrokontroléru

Mapování paměťových sekcí v RAM a Flash

Statická data (linkovací čas):

- `.text`: Strojový kód programu (Flash ROM).
- `.rodata`: Konstanty a řetězcové literály.
- `.data`: Inicializované globální a statické proměnné (kopíruje se z Flash do RAM při startu).
- `.bss`: Neinicializovaná statická data (vynulována při startu systému).

Dynamické sekce za běhu (RAM):

- **Stack (Zásobník):**
  - Lokální proměnné, návratové adresy, kontext přerušení.
  - Roste typicky shora dolů (od nejvyšší adresy k nižší).
- **Heap (Halda):**
  - Oblast pro dynamickou alokaci (funkce `malloc`).
  - Roste zdola nahoru vstříc zásobníku.

Riziko kolize: Stack-Heap Collision

V mikrokontrolérech bez MMU neexistuje hardwarová ochrana paměťových stránek. Pokud se zásobník a halda překryjí, dojde k tichému přepsání dat a fatálnímu pádu.

Rizika dynamické haldy

Proč je `malloc/free` často tabu v embedded praxi

- **1. Paměťová fragmentace (External Fragmentation):**
  - Po sérii alokací a uvolnění různé velikosti vzniká paměťová „mozaika“.
  - Celkový součet volných bajtů může být dostatečný, ale neexistuje žádný **souvislý** blok požadované velikosti → alokace selže.
- **2. Nedeterministická časová složitost:**
  - Doba běhu `malloc()` závisí na prohledávání volných bloků. Nemá garantovaný konstantní čas $O(1)$, což porušuje požadavky na systémy reálného času (Hard Real-Time).
- **3. Bezpečnostní standardy (např. MISRA C):**
  - Nedeterministické chování a riziko vyčerpání paměti vedou k přísným zákazům používání standardní haldy v safety-critical aplikacích.

Deterministické alokační strategie

Fixed-size Block Memory Pool (Bazén fixních bloků)

Princip paměťového bazénu

Předem alokované souvislé pole je rozdělené na bloky stejné velikosti. Volné bloky jsou provázány do jednosměrného spojového seznamu (*free list*) bez extra paměťové režie (ukazatel se uloží přímo do volného bloku).

Výhody:

- **Nulová fragmentace:** Každý uvolněný blok může být okamžitě znovu použit pro jakýkoliv další požadavek.
- **Deterministický čas** $O(1)$**:** Alokace i uvolnění znamenají pouze posun jednoho ukazatele.

Využití v embedded:

- Zprávy v komunikačních frontách (RTOS queues).
- Pakety síťových rozhraní (CAN, Ethernet, BLE).

Implementace Memory Poolu

Deterministická alokace a uvolnění v čase $O(1)$

```
#include <stdint.h>
#include <stddef.h>

// Implicitly stores the 'next' pointer inside the free block itself
typedef struct MemoryBlock {
    struct MemoryBlock *next;
} MemoryBlock;

typedef struct {
    MemoryBlock *free_list;
    // (In practice, also holds a pointer to the raw storage array)
} MemoryPool;

void* pool_alloc(MemoryPool *pool) {
    if (pool->free_list == nullptr) {
        return nullptr; // Out of memory, completely deterministic failure
    }
    MemoryBlock *block = pool->free_list;
    pool->free_list = block->next;
    return (void *)block;
}

void pool_free(MemoryPool *pool, void *ptr) {
    if (ptr == nullptr) return;
```

Arena (Bump) Alokátor

Blesková lineární alokace s hromadným uvolněním

```c
#include <stdint.h>
#include <stddef.h>

typedef struct {
    uint8_t *buffer;
    size_t capacity;
    size_t offset;
} Arena;

void* arena_alloc(Arena *arena, size_t size, size_t alignment) {
    // Calculate memory-aligned offset
    size_t current_addr = (size_t)(arena->buffer + arena->offset);
    size_t aligned_addr = (current_addr + (alignment - 1U)) & ~(alignment - 1U);
    size_t new_offset = (aligned_addr - (size_t)arena->buffer) + size;

    if (new_offset > arena->capacity) {
        return nullptr; // Arena out of capacity
    }

    arena->offset = new_offset;
    return (void *)aligned_addr;
}

// Instant batch deallocation: resets entire state in O(1)
```

Zajištění korektního uvolnění

Prevence úniku paměti (Memory Leaks)

- **Výzva v jazyce C:**
  - Absence automatického *Garbage Collectoru* nebo mechanismů typu RAII (Resource Acquisition Is Initialization) z C++.
  - Zapomenuté uvolnění paměti vede v dlouhoběžících systémech (24/7) k postupnému vyčerpání paměti a následnému pádu nebo restartu (Watchdog reset).

- **Osvědčené přístupy:**
  - **Striktní symetrie**: Modul, který paměť alokuje (např. `sensor_init()`), musí poskytovat i párovou de-inicializační funkci (např. `sensor_deinit()`).
  - **Použití arén**: Jak bylo ukázáno, hromadné uvolnění přes `arena_reset()` eliminuje nutnost sledovat jednotlivé ukazatele.
  - **Rozšíření kompilátorů**: Atribut `__attribute__((cleanup(...)))` podporovaný GCC a Clang pro napodobení chování RAII na konci bloku (scope).

Automatické uvolnění pomocí rozšíření kompilátoru

Atribut cleanup (GCC/Clang) pro imitaci RAII

```c
#include <stdlib.h>
#include <stdio.h>

// Cleanup callback function signature must match the pointer type
void autofree_callback(void *ptr) {
    void **p = (void **)ptr;
    if (*p != nullptr) {
        printf("Automatically freeing memory at exit of scope.\n");
        free(*p);
        *p = nullptr;
    }
}

// Macro helper for cleaner syntax
#define AUTO_FREE __attribute__((cleanup(autofree_callback)))

void process_data_securely(void) {
    // Memory is allocated. The compiler guarantees autofree_callback
    // will be called when 'buffer' goes out of scope.
    AUTO_FREE uint8_t *buffer = malloc(1024);

    if (buffer == nullptr) return; // Safely returns

    // Do work...
```

Samostatný úkol: Deterministický Memory Pool alokátor

Časová dotace: 15–20 minut

Zadání úkolu:
Vytvořte kompletní statický alokátor pevných bloků (*Fixed-size Block Memory Pool*) s časovou složitostí $O(1)$ pro zprávy v RTOS bez použití dynamické haldy (`malloc`/`free`).

Požadavky na implementaci:

1. Definujte strukturu `PoolBlock` (obsahující buď ukazatel `next` ve volném stavu, nebo pole `uint8_t data[32]`) a strukturu `StaticPool` s předalokovaným polem 4 bloků a počítadlem volných bloků.
2. Implementujte funkci `void pool_init(StaticPool *pool)`, která všechny bloky v poli propojí do jednosměrného spojového seznamu volných bloků (`free_list`).
3. Implementujte funkce `void* pool_alloc(StaticPool *pool)` a `void pool_free(StaticPool *pool, void *ptr)` pracující v čase $O(1)$ a ověřte vyčerpání kapacity i opětovné uvolnění bloku.

Hromadná alokace paměti

Hromadná alokace paměti

Indexová vs. pointerová aritmetika a Array Decay

- **Ekvivalence indexace a dereference:**
  - Výraz `a[i]` je kompilátorem striktně definován jako `*(a + i)`.
  - Přičtení čísla 1 k pointeru neposouvá adresu o 1 bajt, nýbrž o `sizeof(*p)` bajtů (krokování podle typu objektu).
- **Rozpad pole na ukazatel (*Array Decay*):**
  - Při předání pole jako argumentu funkce se pole automaticky rozpadá na ukazatel na svůj první prvek (např. `&arr[0]`).
  - Deklarace parametru `void process(uint8_t arr[64])` je syntaktický klam – kompilátor ji interně chápe jako `void process(uint8_t *arr)`.
  - Uvnitř volané funkce je informace o původní velikosti pole nevratně ztracena.
- **Zlaté pravidlo:** Společně s polem předávejte vždy explicitní parametr určující počet prvků (nebo kapacitu bufferu).

Praktická ukázka: Array Decay

Chybný výpočet velikosti vs. bezpečné předání parametru

```c
#include <stdint.h>
#include <stddef.h>

// PITFALL: sizeof inside function evaluates pointer width, not array size!
void process_packet_decay(uint8_t buffer[64]) {
    // On 32-bit MCU: returns 4 bytes (pointer width), NOT 64!
    size_t wrong_size = sizeof(buffer);
    (void)wrong_size;
}

// SAFE APPROACH 1: Explicit length parameter (Universal embedded idiom)
void process_packet_explicit(const uint8_t *buffer, size_t length) {
    for (size_t i = 0; i < length; ++i) {
        // Safe iteration bounded by length
    }
}

// SAFE APPROACH 2: Pointer to fixed-size array (Type safety enforced by compiler)
// Enforces that caller passes exactly an array of 64 bytes.
void process_packet_fixed(const uint8_t (*buffer_ptr)[64]) {
    size_t exact_size = sizeof(*buffer_ptr); // Correctly evaluates to 64 bytes
    (void)exact_size;
}
```

Operátor `sizeof` a paměťové zarovnání

Proč se velikost struktur nerovná součtu velikostí položek

- **Přirozené hardwarové zarovnání (*Data Alignment*):**
  - 32bitový procesor (např. ARM Cortex-M) typicky vyžaduje, aby 32bitové hodnoty začínaly na adrese dělitelné 4.
  - Nezarovnaný přístup (*Unaligned Access*) může vyvolat hardwarovou výjimku (*HardFault / Alignment Fault*) nebo způsobit cykly navíc.
- **Vkládání výplně (*Padding*):**
  - Kompilátor vkládá mezi položky struktury i za její konec neviditelné výplňové bajty, aby zajistil správné zarovnání každého prvku i pro vytvoření pole struktur.
- **Optimalizace pořadí položek:**
  - Řazením položek od hardwarově největšího typu po nejmenší lze ušetřit desítky procent RAM v paměťově omezených mikrokontrolérech a eliminovat zbytečný *padding*.



Praktická ukázka: Padding a Flexible Array Member

Efektivní rozvržení struktur a pakety s proměnnou délkou

```c
#include <stdint.h>
#include <stddef.h>

// BAD LAYOUT: 12 bytes due to padding (1 + 3 pad + 4 + 2 + 2 pad)
struct UnoptimizedSensor {
    uint8_t  id;          // Offset 0
    uint32_t timestamp;   // Offset 4 (3 bytes padding before this)
    uint16_t status;      // Offset 8 (2 bytes trailing padding)
};

// OPTIMIZED LAYOUT: 8 bytes total (No padding needed)
struct OptimizedSensor {
    uint32_t timestamp;   // Offset 0
    uint16_t status;      // Offset 4
    uint8_t  id;          // Offset 6
    uint8_t  reserved;    // Offset 7 (Explicit padding byte for clarity)
};

// FLEXIBLE ARRAY MEMBER (C99/C23): Header with variable-length payload
typedef struct {
    uint16_t packet_id;
    uint16_t payload_len;
    uint8_t  payload[];   // Must be the last member, zero-sized placeholder
} PacketFrame;
```

Kopírování paměti a rizika realokace

Volba správné nízkoúrovňové operace a proč se vyhnout `realloc`

- `memcpy` **vs.** `memmove`**:**
  - `memcpy`: Pouze pro **nepřekrývající se** bloky. Vysoce optimalizované (SIMD, vektorizace).
  - `memmove`: Zvládá **překrývající se** bloky (detekuje směr a kopíruje zepředu či zezadu). Nezbytné pro posuvná okna.

- `memset` **a bezpečnostní limity:**
  - **Optimalizační past**: Kompilátor odstraní „mrtvý“ `memset` na konci životnosti pole. Hesla tak mohou zůstat v RAM! V C23 použijte `memset_explicit`.

- **Rizika** `realloc` **v embedded C:**
  - Pokud nemůže rozšířit paměť na místě, zkopíruje ji jinam a starou uvolní.
  - Pokud paměť dojde, vrátí `nullptr`, **ale stará data neuvolní**. Klasický zápis `p = realloc(p, size)` způsobí okamžitý a nevratný paměťový únik!
  - **Závěr pro RTOS**: `realloc` narušuje časový determinismus. Preferujte zřetězené spojové seznamy nebo dostatečně dimenzované ring buffery.

Praktická ukázka: Bezpečný posun pomocí memmove

Korektní zpracování proudových dat s překryvem

```c
#include <stdint.h>
#include <string.h>
#include <stdbool.h>

#define STREAM_BUFFER_SIZE 128U

typedef struct {
    uint8_t storage[STREAM_BUFFER_SIZE];
    size_t valid_bytes;
} StreamBuffer;

// Consume N bytes from the front and shift remaining data to index 0
bool stream_buffer_consume(StreamBuffer *stream, size_t consumed_bytes) {
    if (consumed_bytes > stream->valid_bytes) {
        return false; // Bounds protection
    }

    size_t remaining_bytes = stream->valid_bytes - consumed_bytes;

    if (remaining_bytes > 0) {
        // CRITICAL: Source and destination overlap! memcpy would trigger UB.
        // memmove safely preserves remaining incoming stream bytes.
        memmove(&stream->storage[0],
                &stream->storage[consumed_bytes],
```

Samostatný úkol: Optimalizace paddingu a posuvné okno

Časová dotace: 15–20 minut

Zadání úkolu:
Ušetřete paměť RAM přeuspořádáním položek telemetrické struktury a implementujte bezpečné posuvné okno (*sliding window*) nad proudovým bufferem s překrývající se pamětí.

Požadavky na implementaci:

1. Mějme neoptimalizovanou strukturu `BadHeader` s položkami v pořadí: `uint8_t flags`, `uint64_t timestamp`, `uint8_t id`, `uint32_t crc`, `uint16_t len`. Vytvořte strukturu `GoodHeader` se stejnými položkami seřazenými tak, aby měla nulový vnitřní padding, a porovnejte jejich `sizeof`.
2. Implementujte funkci `void sliding_window_append(uint8_t *win, size_t win_size, uint8_t new_sample)`, která pomocí `memmove` posune stávající bajty v poli o 1 pozici doleva a na poslední index zapíše nový vzorek.

Dynamické datové struktury pro embedded

Dynamické datové struktury v embedded

Proč klasické učebnicové implementace na MCU selhávají

- **Problém klasických struktur (stromy, spojové seznamy na haldě):**
  - Každý prvek alokován samostatně přes `malloc()` → masivní fragmentace RAM.
  - **Paměťová režie ukazatelů:** Na 32bitovém MCU zabírá obousměrný ukazatel $2 \times 4 = 8$ bajtů. Pro 2bajtový senzorový údaj tvoří metadata 80 % velikosti uzlu!
  - **Cache missy:** Dynamicky alokované uzly jsou rozptýleny po celé RAM, což znemožňuje efektivní přednačítání do cache sběrnice.
- **Embedded paradigma: Bounded Data Structures**
  - Všechny struktury mají striktně definovanou **maximální kapacitu** známou v době překladu nebo inicializace.
  - Paměť je alokována souvisle ve statické sekci (`.bss`) nebo v paměťovém bazénu.
  - Deterministické chování při vyčerpání kapacity namísto pádu systému.

Bounded Dynamic Array

Rostoucí sekvence s fixním paměťovým stropem

```c
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

#define SENSOR_ARRAY_MAX_CAPACITY 32U

typedef struct {
    int16_t temperature_raw;
    uint32_t timestamp_ms;
} SensorRecord;

typedef struct {
    SensorRecord items[SENSOR_ARRAY_MAX_CAPACITY];
    size_t count;
} BoundedSensorArray;

bool bounded_array_push(BoundedSensorArray *array, SensorRecord record) {
    if (array->count >= SENSOR_ARRAY_MAX_CAPACITY) {
        return false; // Deterministic overflow rejection (No silent heap crash)
    }
    array->items[array->count] = record;
    array->count++;
    return true;
}
```

Rizika funkce `realloc` v embedded

Skrytá alokace, fragmentace a ztráta původního ukazatele

- **Jak interně funguje `realloc(ptr, new_size)`:**
  1. Pokusí se rozšířit stávající paměťový blok na místě.
  2. Pokud za blokem není volné místo, prohledá haldu a alokuje **zcela nový blok**.
  3. Zkopíruje stará data do nového umístění (`memcpy`) a starý blok uvolní.

- **Fatální pasti v embedded kódu:**
  - **Tichý únik paměti při chybě**: Pokud alokace selže, `realloc` vrátí `nullptr`, ale **původní blok neuvolní**! Zápis `p = realloc(p, size)` okamžitě ztratí referenci na původní data.
  - **Časový nedeterminismus**: Kopírování velkých sekvencí může způsobit zpoždění, které poruší reálný čas (Real-Time deadlines).

- **Doporučení pro embedded:** Používat buď statické stropy, nebo blokovou (chunked) alokaci z Memory Poolu s pevnou velikostí stránky.

Kruhový buffer pro proměnnou délku zpráv

Paketové FIFO s kontrolou celistvosti

```c
#include <stdint.h>
#include <stdbool.h>
#include <string.h>

#define PACKET_STORAGE_SIZE 256U
#define PACKET_STORAGE_MASK (PACKET_STORAGE_SIZE - 1U)

typedef struct {
    uint8_t buffer[PACKET_STORAGE_SIZE];
    uint32_t write_idx;
    uint32_t read_idx;
} PacketRingBuffer;

bool packet_rb_push(PacketRingBuffer *rb, const uint8_t *payload, uint8_t len) {
    uint32_t free_space = PACKET_STORAGE_SIZE - (rb->write_idx - rb->read_idx);
    if (free_space < (uint32_t)(len + 1U)) {
        return false; // Not enough contiguous space for length header + payload
    }
    // Write 1-byte length header followed by message bytes
    rb->buffer[rb->write_idx & PACKET_STORAGE_MASK] = len;
    rb->write_idx++;
    for (uint8_t i = 0; i < len; ++i) {
        rb->buffer[rb->write_idx & PACKET_STORAGE_MASK] = payload[i];
        rb->write_idx++;
```

Index-based Linked List

Spojový seznam bez ukazatelů a bez použití haldy

Princip indexovaného seznamu

Všechny uzly jsou předalokovány v jednom statickém poli. Ukazatele na další/předchozí uzel jsou nahrazeny celočíselnými indexy (`int16_t` nebo `uint8_t`). Hodnota `-1` reprezentuje `nullptr`.

Zásadní výhody:

- **Poloviční paměťová režie:** Index zabírá 1–2 bajty namísto 4 bajtů ukazatele.
- **Nulová fragmentace:** Žádné volání `malloc()` / `free()`.
- **Serializovatelnost:** Data lze poslat po CAN/DMA bez přepočítávání ukazatelů!

Správa volných uzlů:

- Volné pozice v poli jsou samy provázány do *Free Listu*.
- Vložení i smazání uzlu probíhá v garantovaném deterministickém čase $O(1)$.

Praktická ukázka: Index-based List

Implementace statického seznamu s provázaným Free Listem

```c
#include <stdint.h>
#include <stdbool.h>

#define MAX_NODES 16
#define LIST_INVALID_INDEX -1

typedef struct {
    uint32_t sensor_value;
    int16_t next; // Index to next node in array (replaces pointer)
} StaticNode;

typedef struct {
    StaticNode pool[MAX_NODES];
    int16_t head;
    int16_t free_head;
} StaticLinkedList;

void static_list_init(StaticLinkedList *list) {
    list->head = LIST_INVALID_INDEX;
    list->free_head = 0;
    // Link all initial nodes into the free list
    for (int16_t i = 0; i < (MAX_NODES - 1); ++i) {
        list->pool[i].next = (int16_t)(i + 1);
    }
```

Samostatný úkol: Mazání uzlu v Index-based seznamu

Časová dotace: 15–20 minut

Zadání úkolu:
Rozšiřte implementaci statického indexovaného spojového seznamu (`StaticLinkedList` ze slidu 71) o funkci pro odebrání prvního prvku (`pop_front`) a navrácení uvolněného indexu zpět do `free_head` v čase $O(1)$.

Požadavky na implementaci:

1. Vyjděte ze struktury `StaticLinkedList` s polem `pool[MAX_NODES]`, indexem aktivního začátku `head` a indexem volných uzlů `free_head`.
2. Implementujte funkci `bool static_list_pop_front(StaticLinkedList *list, uint32_t *out_value)`:
   - Pokud je seznam prázdný (`head == LIST_INVALID_INDEX`), vraťte `false`.
   - Uložte hodnotu z čelního uzlu do `*out_value` a posuňte `head` na další uzel.
   - Odebraný index zařaďte zpět na začátek seznamu volných uzlů (`free_head`), aby nedocházelo k vyčerpání kapacity bazénu.

Specifika embedded programování

Klíčové slovo `volatile`

Základní kámen pro memory-mapped I/O

- **Význam klíčového slova `volatile`:**
  - Instruuje kompilátor, že hodnota proměnné se může kdykoliv změnit zásahem zvenčí (hardware, přerušení, jiné vlákno).
  - **Zabraňuje optimalizacím**: Kompilátor nesmí přístup k této proměnné vynechat, přeskládat nebo hodnotu trvale uložit do registru procesoru (tzv. caching). Každé čtení/zápis musí jít fyzicky na sběrnici.

- **Memory-mapped I/O (MMIO):**
  - Většina mikrokontrolérů (např. ARM Cortex-M) ovládá periferie čtením a zápisem na specifické adresy v paměti.
  - Zápis na adresu nerozsvítí jen buňku v RAM, ale fyzicky překlopí pin nebo odešle znak po UARTu. Bez `volatile` by optimalizátor opakované zápisy na stejnou adresu (např. odesílání stringu) sloučil do jednoho.

Praktická ukázka: Mapování registrů

Definice hardwarové periferie pomocí struktur a volatile

```c
#include <stdint.h>

// Definition of a hardware peripheral (e.g., General Purpose I/O)
// All registers interacting with hardware MUST be volatile
typedef struct {
    volatile uint32_t MODE;   // Mode register (Input/Output)
    volatile uint32_t OUTPUT; // Output data register
    volatile uint32_t INPUT;  // Input data register
} GpioPeripheral;

// Memory address defined by the microcontroller's datasheet
#define GPIO_PORT_A_BASE 0x40020000UL

// Cast the raw address to our struct pointer
#define GPIOA ((GpioPeripheral *)GPIO_PORT_A_BASE)

void hardware_toggle_pin(void) {
    // 1. Set pin 5 as output (assuming binary manipulation)
    GPIOA->MODE |= (1UL << 5);

    // 2. Toggle pin 5 high. Compiler WILL NOT optimize this away.
    GPIOA->OUTPUT |= (1UL << 5);
}
```

Samostatný úkol: Ovladač UART periferie (MMIO)

Časová dotace: 15–20 minut

Zadání úkolu:
Navrhněte strukturu paměťově mapovaných registrů (MMIO) pro hardwarovou jednotku UART a implementujte neoptimalizovatelnou smyčku pro odeslání znakového řetězce.

Požadavky na implementaci:

1. Definujte strukturu `UartRegisters` obsahující tři 32bitové registry opatřené kvalifikátorem `volatile`: `STATUS` (stavový registr), `TX_DATA` (vysílací registr) a `CONTROL` (řídicí registr).
2. Definujte bitové masky: `UART_STATUS_TX_EMPTY` (bit 7) a `UART_CTRL_ENABLE` (bit 0).
3. Implementujte funkci `bool uart_send_byte_timeout(UartRegisters *uart, uint8_t byte, uint32_t max_polls)`, která v aktivní smyčce (*polling*) čeká na nahození příznaku `TX_EMPTY` (maximálně `max_polls` iterací) a poté zapíše bajt do `TX_DATA`.

Přerušení (ISR) a sdílená data

Kontext přerušení (ISR) vs. kontext úlohy

Zásadní omezení a specifika běhu v obsluze přerušení

- **Asynchronní spuštění:**
  - Obsluha přerušení (*Interrupt Service Routine* – ISR) je spouštěna hardwarovou událostí bez ohledu na aktuální stav hlavní smyčky nebo RTOS tasku.
  - V architektuře ARM Cortex-M dochází k automatickému uložení kontextu na zásobník (registry R0–R3, R12, LR, PC, xPSR).
- **Oddělení zásobníků (Dual Stack Pointer):**
  - **MSP (Main Stack Pointer)**: Využíván pro systémový start a veškeré ISR handlery.
  - **PSP (Process Stack Pointer)**: Využíván konkrétními aplikačními úlohami v RTOS.
- **Kardinální pravidla pro kód uvnitř ISR:**
  - **Žádné blokování**: Zákaz čekání na mutexy, volání zpoždění (`sleep`) a dlouhých polling smyček.
  - **Žádná obecná alokace**: Nepoužívat funkce jako `malloc` ani knihovní I/O (`printf`).
  - **Minimální délka**: Rychle odbavit hardware, uložit data do fronty a předat zpracování aplikační vrstvě.

Souběh (Race Conditions) a kritické sekce

Riziko neatomického přístupu ke sdílenému stavu

Definice souběhu

K souběhu dochází, pokud hlavní program a obsluha přerušení přistupují ke stejné proměnné, přičemž alespoň jeden z nich provádí zápis a operace není atomická.

- **Problém vícebajtových proměnných:**
  - Na 32bitovém MCU je čtení 64bitového celého čísla (`uint64_t`) přeloženo do dvou samostatných instrukcí `LDR`.
  - Pokud ISR zasáhne mezi prvním a druhým čtením, přečte hlavní program spodní polovinu staré hodnoty a horní polovinu nové hodnoty (*torn read*).
- **Kritická sekce maskováním přerušení:**
  - Dočasné potlačení přerušení na úrovni jádra CPU.
  - **Chyba začátečníků**: Pouhé volání `__disable_irq()` a `__enable_irq()` – nepodporuje vnořování a nekorektně zapne přerušení uvnitř již existující kritické sekce!

Praktická ukázka: Bezpečná kritická sekce

Uložení a obnova masky přerušení (RAII vzor v C)

```c
#include <stdint.h>

// Read and restore CPU interrupt mask register (PRIMASK on ARM Cortex-M)
static inline uint32_t enter_critical_section(void) {
    uint32_t primask;
    __asm__ volatile("mrs %0, primask" : "=r"(primask));
    __asm__ volatile("cpsid i" : : : "memory");
    return primask; // Return original state before masking
}

static inline void exit_critical_section(uint32_t primask) {
    // Restore previous interrupt state (safe for nested calls)
    __asm__ volatile("msr primask, %0" : : "r"(primask) : "memory");
}

static volatile uint64_t system_uptime_us = 0;

uint64_t get_system_uptime(void) {
    // 64-bit read requires two 32-bit operations on Cortex-M
    uint32_t status = enter_critical_section();
    uint64_t current_time = system_uptime_us;
    exit_critical_section(status);
    return current_time;
}
```

Atomické operace (`stdatomic.h`)

Bezzámková synchronizace přímo na úrovni instrukcí procesoru

- Zavedeno ve standardu C11 a rozšířeno v C23 (`<stdatomic.h>`).
- **Hardwarový princip (ARM Exclusive Monitors):**
  - Místo vypínání přerušení využívá procesor dvojici instrukcí `LDREX` a `STREX`.
  - `LDREX` načte hodnotu a označí paměťovou adresu jako sledovanou.
  - `STREX` provede zápis pouze tehdy, pokud od čtení nedošlo k žádnému zásahu do monitorované adresy (jinak selže a smyčka se opakuje).
- **Výhody oproti vypínání přerušení:**
  - Neovlivňuje latenci přerušení s vysokou prioritou.
  - Zápis probíhá bezzámkově a bezpečně i mezi více jádry (Multicore MCU).
- **Paměťové modely (*Memory Ordering*):**
  - `memory_order_relaxed`: Pouze atomická hodnota, bez garance pořadí okolních zápisů.
  - `memory_order_acquire` / `memory_order_release`: Garantuje pořadí přenosu dat mezi úlohami.

Praktická ukázka: Atomické operace v C23

Zpracování událostí bez vypínání přerušení

```c
#include <stdatomic.h>
#include <stdbool.h>
#include <stdint.h>

// Lock-free atomic counter shared between ISR and worker task
static atomic_uint_fast32_t isr_event_counter = ATOMIC_VAR_INIT(0);
static atomic_flag packet_ready_flag = ATOMIC_FLAG_INIT;

// Executed in hardware interrupt context (e.g. GPIO button interrupt)
void exti_gpio_irq_handler(void) {
    // Atomic increment without disabling global interrupts
    atomic_fetch_add_explicit(&isr_event_counter, 1U, memory_order_relaxed);

    // Set flag indicating that a new event requires processing
    atomic_flag_clear_explicit(&packet_ready_flag, memory_order_release);
}

// Executed in background task context
void worker_task_loop(void) {
    // Atomically read counter value
    uint32_t count = atomic_load_explicit(&isr_event_counter, memory_order_relaxed);
    if (count > 0) {
        // Process collected events
    }
```

Kruhový buffer pro ISR a Task

Single-Producer Single-Consumer (SPSC) architektura

Princip SPSC Ring Bufferu

Jednosměrná fronta, kde data zapisuje výhradně jeden producent (např. UART RX přerušení) a vybírá výhradně jeden konzument (aplikační smyčka).

Proč nepotřebuje zámky:

- Producent mění **pouze** ukazatel zápisu (`head`).
- Konzument mění **pouze** ukazatel čtení (`tail`).
- Oba ukazatele jsou vzájemně nezávislé – nedochází ke kolizi zápisů!

Pravidla návrhu:

- Velikost bufferu volit jako mocninu dvou (2^N) $\rightarrow$ zrychlení indexace pomocí bitové masky `head & (SIZE - 1)`.
- Paměťová bariéra před posunem indexu pro zajištění viditelnosti dat.

Praktická ukázka: Bezzámkový SPSC FIFO

Bezpečný přenos dat z UART přerušení do aplikace

```c
#include <stdint.h>
#include <stdbool.h>

#define SPSC_BUFFER_SIZE 64U // Must be a power of two
#define SPSC_BUFFER_MASK (SPSC_BUFFER_SIZE - 1U)

typedef struct {
    uint8_t storage[SPSC_BUFFER_SIZE];
    volatile uint32_t head; // Written only by Producer (ISR)
    volatile uint32_t tail; // Written only by Consumer (Task)
} SpscRingBuffer;

bool spsc_push_from_isr(SpscRingBuffer *rb, uint8_t byte) {
    uint32_t current_head = rb->head;
    uint32_t next_head = (current_head + 1U) & SPSC_BUFFER_MASK;
    if (next_head == rb->tail) {
        return false; // Buffer full (overflow protection)
    }
    rb->storage[current_head] = byte;
    __asm__ volatile("" ::: "memory"); // Compiler barrier: data must be stored before head moves
    rb->head = next_head;
    return true;
}
```

Samostatný úkol: Bezzámkový SPSC buffer pro ADC vzorky

Časová dotace: 15–20 minut

Zadání úkolu:
Implementujte bezzámkovou frontu (*Single-Producer Single-Consumer Ring Buffer*) pro přenos 16bitových vzorků z přerušení časovače/ADC do aplikační úlohy spolu s atomickým počítadlem zahozených vzorků.

Požadavky na implementaci:

1. Definujte strukturu `AdcRingBuffer` s kapacitou 8 prvků (mocnina dvou), polem `uint16_t samples[8]` a indexy `volatile uint32_t head` a `tail`.
2. Definujte atomické počítadlo `atomic_uint_fast32_t dropped_samples` z hlavičky `<stdatomic.h>`.
3. V obsluze `adc_isr_push` vložte vzorek pomocí maskování `(head + 1U) \& MASK` a paměťové bariéry. Pokud je buffer plný, atomicky inkrementujte `dropped_samples` (`memory\_order\_relaxed`).

Úlohy a synchronizace v RTOS

Úlohy v RTOS (Tasks)

Kontext úlohy, Task Control Block a přepnutí kontextu

- **Anatomie úlohy (Task):**
  - Samostatná prováděcí jednotka běžící v nekonečné smyčce.
  - Reprezentována strukturou **TCB (Task Control Block)**, která uchovává aktuální ukazatel na zásobník (SP), stav (Ready, Running, Blocked, Suspended) a prioritu.
- **Mechanismus přepnutí kontextu (Context Switch na ARM Cortex-M):**
  - Využívá softwarové přerušení s nejnižší prioritou: **PendSV**.
  - Hardwarové uložení základních registrů na zásobník úlohy (PSP).
  - Softwarové uložení zbývajících registrů jádra (R4–R11) v assembleru PendSV handleru.
  - Přepnutí ukazatele TCB na novou úlohu a obnova jejích registrů ze zásobníku.
- **Oddělení zásobníků:** Každá úloha má striktně vyhrazený vlastní paměťový blok. Přetečení zásobníku jedné úlohy přepíše paměť sousedních struktur.

Statická alokace úlohy a monitoring paměti

FreeRTOS bez dynamické haldy a diagnostika přetečení

```c
#include <stdint.h>
#include "FreeRTOS.h"
#include "task.h"

#define SENSOR_TASK_STACK_WORDS 256U

// Statically allocated memory for stack and TCB (Zero heap usage)
static StackType_t sensor_task_stack[SENSOR_TASK_STACK_WORDS];
static StaticTask_t sensor_task_tcb;

void sensor_task_entry(void *argument) {
    (void)argument;
    for (;;) {
        // Periodic hardware measurement
        vTaskDelay(pdMS_TO_TICKS(100));

        // Diagnostics: Check minimum unused stack space ever recorded
        UBaseType_t min_stack_headroom = uxTaskGetStackHighWaterMark(nullptr);
        if (min_stack_headroom < 32U) {
            // WARNING: Stack utilization exceeded safe threshold!
        }
    }
}
```

Synchronizační primitiva

Zásadní rozdíl: Mutex vs. Binární semafor

Vlastnost	Mutex (Vzájemné vyloučení)	Binární semafor (Signalizace)
Koncept vlastnictví	Má vlastníka (kdo zamkne, musí odemknout)	Nemá vlastníka (jeden dává, druhý bere)
Primární účel	Ochrana sdíleného zdroje (Resource lock)	Synchronizace událostí (Signaling)
Použití v ISR	**PŘÍSNĚ ZAKÁZÁNO**	**POVOLENO** (`FromISR` varianty)
Inverze priorit	Obsahuje **Priority Inheritance**	**Nemá** Priority Inheritance
Počáteční stav	Odemčen (k dispozici)	Typicky prázdný (čeká na událost)

Častá architektonická chyba

Použití binárního semaforu pro ochranu sdílené sběrnice (např. SPI). Pokud nastane inverze priorit, binární semafor ji nedokáže vyřešit a systém uvízne!

Praktická ukázka: Ochrana periferie mutexem

Eliminace uvíznutí pomocí deterministických timeoutů

```
#include <stdbool.h>
#include "FreeRTOS.h"
#include "semphr.h"

static SemaphoreHandle_t i2c_bus_mutex = nullptr;

bool i2c_bus_read_sensor(uint8_t dev_addr, uint8_t *dest, size_t length) {
    // CRITICAL: In real-time systems, never block indefinitely with portMAX_DELAY!
    const TickType_t wait_timeout = pdMS_TO_TICKS(50);

    // Attempt to acquire exclusive ownership of peripheral bus
    if (xSemaphoreTake(i2c_bus_mutex, wait_timeout) != pdTRUE) {
        return false; // Bus busy timeout: Allow caller to gracefully recover
    }

    // Critical section: Access hardware registers safely
    bool transfer_ok = hardware_i2c_transfer(dev_addr, dest, length);

    // Always release ownership immediately after use
    xSemaphoreGive(i2c_bus_mutex);
    return transfer_ok;
}
```

Inverze priorit (Priority Inversion)

Mechanismus selhání a záchrana pomocí Priority Inheritance

Scénář nekontrolované inverze:

1. **Task L** (nízká priorita) zamkne mutex sdíleného senzoru.
2. **Task H** (vysoká priorita) se probudí a pokusí se získat mutex → je zablokován.
3. **Task M** (střední priorita), který mutex nepotřebuje, se probudí a **předběhne běžící Task L**!
4. **Důsledek:** Task H nepřímo čeká na Task M, což porušuje veškeré časové garance (Mars Pathfinder, 1997).

Řešení: Priority Inheritance:

- Jakmile Task H začne čekat na mutex držený Taskem L, RTOS kernel **dočasně zvýší prioritu Tasku L na úroveň Tasku H**.
- Task M nemůže Task L přerušit.
- Task L rychle dokončí kritickou sekci, uvolní mutex a jeho priorita klesne zpět.
- Task H okamžitě získává mutex a pokračuje v běhu.

Uváznutí (Deadlock) a hierarchické zamykání

Prevence uváznutí striktním pořadím zámků

```c
#include "FreeRTOS.h"
#include "semphr.h"

static SemaphoreHandle_t flash_mutex = nullptr;
static SemaphoreHandle_t uart_mutex = nullptr;

// DEADLOCK RISK:
// Task A locks Flash, then attempts to lock UART.
// Task B locks UART, then attempts to lock Flash.
// Result: Mutual circular wait where neither task can ever continue!

// SAFE ARCHITECTURE: Strict Global Lock Hierarchy
// Rule: Always acquire flash_mutex BEFORE uart_mutex system-wide!
void log_critical_telemetry(const uint8_t *payload, size_t size) {
    if (xSemaphoreTake(flash_mutex, pdMS_TO_TICKS(100)) == pdTRUE) {
        if (xSemaphoreTake(uart_mutex, pdMS_TO_TICKS(50)) == pdTRUE) {
            // Write to both storage and communication channels safely
            xSemaphoreGive(uart_mutex);
        }
        xSemaphoreGive(flash_mutex);
    }
}
```

Event Groups (Událostní skupiny)

Atomické čekání na kombinace asynchronních událostí

```
#include "FreeRTOS.h"
#include "event_groups.h"

#define EVENT_BIT_WIFI_CONNECTED   (1UL << 0)
#define EVENT_BIT_TIME_SYNCHRONIZED (1UL << 1)
#define EVENT_BIT_ALL_READY         (EVENT_BIT_WIFI_CONNECTED | EVENT_BIT_TIME_SYNCHRONIZED)

static EventGroupHandle_t network_event_group = nullptr;

void cloud_sync_task(void *argument) {
    (void)argument;
    for (;;) {
        // Wait atomically for BOTH conditions before starting upload:
        // xWaitForAllBits = pdTRUE, xClearOnExit = pdFALSE
        EventBits_t status = xEventGroupWaitBits(
            network_event_group,
            EVENT_BIT_ALL_READY,
            pdFALSE, pdTRUE, portMAX_DELAY
        );

        if ((status & EVENT_BIT_ALL_READY) == EVENT_BIT_ALL_READY) {
            // Safe to proceed with encrypted HTTPS cloud payload upload
        }
    }
```

Samostatný úkol: Prevence Deadlocku a Event Groups

Časová dotace: 15–20 minut

Zadání úkolu:
Opravte chybný návrh dvou úloh v RTOS, které způsobují vzájemné uváznutí (*Deadlock*) při současném přístupu na sběrnici SPI a do interní EEPROM, a doplňte synchronizaci startu přes *Event Group*.

Požadavky na implementaci:

1. Definujte globální hierarchii zámků: `spi_mutex` (úroveň 1) musí být v celém systému vždy uzamčen **před** `eeprom_mutex` (úroveň 2).
2. Implementujte funkci `bool safe_backup_sensor_to_eeprom(void)`, která zamkne oba mutexy ve správném pořadí s konečným timeoutem a po zápisu je v opačném pořadí uvolní.
3. Implementujte kontrolu příznaků `EVENT_ADC_CALIBRATED` (bit 0) a `EVENT_STORAGE_MOUNTED` (bit 1), která povolí zálohování pouze tehdy, jsou-li oba bity současně aktivní.

Cache, DMA a zarovnání dat

Direct Memory Access (DMA)

Asynchronní přenos dat na pozadí výpočtu CPU

- **Princip řadiče DMA:**
  - Samostatný hardwarový modul fungující jako **Bus Master** na systémové sběrnici (AHB/AXI).
  - Přenáší data mezi periferií a pamětí (případně pamětí a pamětí) bez instrukční asistence CPU.
  - Uvolňuje jádro procesoru pro výpočetní úlohy a šetří spotřebu energie.
- **Režimy přenosu DMA:**
  - **Normal mode**: Jednorázový přenos definovaného počtu bloků se zakončením přerušením (*Transfer Complete*).
  - **Circular mode**: Kruhové kontinuální plnění bufferu (typické pro audio I2S, ADC vzorkování a UART příjem s využitím *Half-Transfer* přerušení).
- **Architektonická výzva:** Jakmile procesor zavede vyrovnávací paměť (Cache), DMA i CPU přistupují ke stejné paměti dvěma různými cestami.

Vyrovnávací paměť (Cache) na Cortex-M7/M33

Struktura Cache Line a zásady zápisu

- **Hierarchie paměti:**
  - CPU pracuje na vysoké frekvenci (stovky MHz), zatímco externí či interní SRAM sběrnice je pomalejší.
  - **L1 D-Cache (Data Cache)**: Uchovává kopie nedávno použitých paměťových bloků.
  - Základní jednotkou přenosu je **Cache Line** (na ARM Cortex-M7 má pevnou velikost **32 bajtů**).
- **Zápisové strategie:**
  - **Write-Through**: Každý zápis jde do cache i okamžitě do fyzické RAM (bezpečnější, ale pomalejší).
  - **Write-Back**: Zápis se provede pouze do cache a řádek se označí jako *Dirty*. Do RAM se data zapíší až při vyklizení řádku (*Eviction*).
- **Problém koherence (Cache Incoherency):** Řadič DMA obchází L1 Cache jádra a komunikuje přímo s fyzickou RAM.

Operace údržby Cache

Řešení koherence pro vysílání (TX) a příjem (RX)

Směr CPU → DMA (Vysílání / TX):

1. CPU sestaví paket v RAM (data uvíznou v D-Cache jako *dirty*).
2. DMA začne číst přímo z RAM → odešle **stará, neplatná data**!

Řešení: Operace Clean (Flush)

- Vynutí okamžitý zápis *dirty* řádků z D-Cache do fyzické RAM před spuštěním DMA.

Směr DMA → CPU (Příjem / RX):

1. DMA zapíše nový paket přímo do fyzické RAM.
2. CPU se pokusí přečíst data, ale načte **stará cached data** z D-Cache!

Řešení: Operace Invalidate

- Zneplatní dané řádky v D-Cache. Následné čtení procesorem je vynuceno přímo z čerstvé RAM.

Praktická ukázka: CMSIS Cache údržba

Korektní sekvence volání před a po DMA transferu

```c
#include <stdint.h>
#include <stddef.h>

// Standard CMSIS core functions for ARM Cortex-M7
void SCB_CleanDCache_by_Addr(uint32_t *addr, int32_t dsize);
void SCB_InvalidateDCache_by_Addr(uint32_t *addr, int32_t dsize);

void dma_start_transmit(const uint8_t *tx_buffer, size_t length) {
    // 1. Flush modified CPU data from L1 D-Cache down into physical RAM
    SCB_CleanDCache_by_Addr((uint32_t *)tx_buffer, (int32_t)length);

    // 2. Safely trigger hardware DMA channel
    hardware_dma_start_tx(tx_buffer, length);
}

void dma_rx_complete_isr_callback(uint8_t *rx_buffer, size_t length) {
    // 1. Invalidate stale cache lines so CPU is forced to fetch fresh RAM
    SCB_InvalidateDCache_by_Addr((uint32_t *)rx_buffer, (int32_t)length);

    // 2. CPU safely reads received hardware data
    process_incoming_packet(rx_buffer, length);
}
```

Riziko: False Sharing a ztráta dat

Proč musí být DMA buffery striktně zarovnané

Scénář tiché destrukce dat (Data Corruption)

Pokud buffer pro příjem DMA nezačíná a nekončí přesně na hranici Cache Line (32 bajtů), sdílí stejný řádek s okolními proměnnými aplikace!

- **Jak dojde ke katastrofě:**
  1. CPU modifikuje sousední proměnnou (např. `system_state`), která leží na stejné 32B cache line jako DMA buffer. Hodnota je v cache označena jako *dirty*.
  2. DMA přijme paket po sběrnici do RAM.
  3. Ovladač zavolá `SCB_InvalidateDCache_by_Addr` pro DMA buffer.
  4. **Důsledek**: Cache zneplatní celých 32 bajtů – **včetně proměnné** `system_state`!
  5. Změny provedené jádrem v sousední proměnné jsou navždy zničeny a ztraceny.
- **Zákon pro DMA buffery:** Velikost i adresa bufferu musí být násobkem velikosti Cache Line (32 **bajtů**).

Zarovnání bufferu pomocí standardu C

Vynucení hranice Cache Line pomocí alignas

```c
#include <stdint.h>
#include <stdalign.h> // Standard C11 / C23 keyword alignas

#define CACHE_LINE_SIZE 32U

// Macro to round up buffer size to the nearest multiple of cache line size
#define ALIGN_UP_CACHE(size) \
    (((size) + (CACHE_LINE_SIZE - 1U)) & ~(CACHE_LINE_SIZE - 1U))

#define RAW_PAYLOAD_SIZE 70U
#define SAFE_DMA_BUFFER_SIZE ALIGN_UP_CACHE(RAW_PAYLOAD_SIZE) // Evaluates to 96 bytes

// Buffer is strictly aligned to 32-byte boundary and padded to full lines
// Prevents any adjacent stack/static variables from sharing the cache line!
alignas(CACHE_LINE_SIZE) static uint8_t dma_rx_storage[SAFE_DMA_BUFFER_SIZE];

void setup_safe_reception(void) {
    // Safe to invalidate entire SAFE_DMA_BUFFER_SIZE without corrupting neighboring data
    hardware_dma_start_rx(dma_rx_storage, SAFE_DMA_BUFFER_SIZE);
}
```

Alternativa: Non-cacheable paměť a MPU

Úplná eliminace nutnosti manuální správy cache

- **Nastavení MPU (Memory Protection Unit):**
  - Nakonfigurovat vyhrazenou oblast RAM (např. SRAM4 na STM32H7) jako **Device** nebo **Normal**, **Non-Cacheable**.
  - Každý přístup CPU i DMA jde přímo na sběrnici RAM bez jakékoli cache mezivrstvy.

```c
#include <stdint.h>

// Place DMA descriptors and circular buffers into dedicated non-cacheable RAM
// Section is defined in the microcontroller linker script (.ld)
#define DMA_BUFFER_SECTION __attribute__((section(".noncacheable_sram"), aligned(32)))

// Zero cache-maintenance overhead: Hardware coherency is guaranteed by MPU
DMA_BUFFER_SECTION static uint8_t network_dma_ring_buffer[1024];

void init_zero_overhead_dma(void) {
    // No SCB_CleanDCache or SCB_InvalidateDCache needed for this buffer!
    hardware_ethernet_dma_init(network_dma_ring_buffer, sizeof(network_dma_ring_buffer));
}
```

Samostatný úkol: Ochrana DMA deskriptorů před False Sharingem

Časová dotace: 15–20 minut

Zadání úkolu:
Zajistěte, aby při příjmu síťového rámce přes DMA na procesoru ARM Cortex-M7 nedošlo k přepsání sousedních proměnných při operaci `SCB_InvalidateDCache_by_Addr` (*False Sharing*).

Požadavky na implementaci:

1. Definujte velikost řádku cache `CACHE_LINE_SIZE 32U` a makro `ALIGN_UP_32(sz)`, které libovolnou velikost zaokrouhlí nahoru na nejbližší násobek 32 bajtů.
2. Alokujte statický přijímací buffer pro 100bajtovou zprávu tak, aby jak jeho počáteční adresa (`alignas(32)`), tak jeho celková velikost byly striktně zarovnány na 32B hranici.
3. Napište diagnostickou funkci `bool verify_dma_buffer_safety(const void *ptr, size_t size)`, která programově zkontroluje zarovnání adresy i velikosti před spuštěním DMA přenosu.

Struktury a abstrakce

Od „C jako assembleru“ k softwarové architektuře

Změna paradigmatu v embedded programování

- **Zlozvyk procedurálního přístupu:**
  - Začínající embedded programátoři často píší kód jako „lepší assembler“ – hardware se ovládá přímým zápisem do registrů z různých míst programu.
  - Stav systému je rozprostřen do desítek nezávislých globálních proměnných.
- **Problémy globálního stavu:**
  - Kód nelze snadno testovat (Unit Testing je závislý na stavu hardwaru).
  - Nemožnost znovupoužití: Pokud systém dostane druhý stejný senzor, je nutné kód zduplikovat a přepsat názvy proměnných.
- **Řešení pomocí struktur:**
  - Jazyk C sice není objektově orientovaný, ale `struct` umožňuje vynikající zapouzdření stavu.
  - Namísto globálních proměnných předáváme funkcím ukazatel na kontextovou strukturu (tzv. *Context Pointer* idiom).

Praktická ukázka: Globální stav vs. Kontextový ukazatel

Umožnění vícenásobné instanciace

```c
#include <stdint.h>
#include <stdbool.h>

// ANTI-PATTERN: Hardware-centric approach (Global variables)
uint32_t g_encoder_ticks = 0;
bool     g_encoder_direction = true;
void process_encoder_global(void) { /* Mutates globals directly */ }

// ARCHITECTURAL APPROACH: Software-centric (Encapsulated state)
typedef struct {
    uint32_t ticks;
    bool     direction;
    uint32_t hardware_base_address; // Abstraction of physical HW
} EncoderContext;

// Functions operate purely on the provided context
void encoder_process(EncoderContext *ctx) {
    if (ctx == nullptr) return;
    // Process data updating only the specific instance
    ctx->ticks++;
}
```

Seskupení závislých dat a Invarianty

Ochrana vnitřní konzistence datových struktur

- **Co je to Invariant:**
  - Logická podmínka nebo vztah mezi proměnnými, který musí platit po celou dobu existence objektu (s výjimkou průběhu samotné modifikující funkce).
  - Příklad: „*Hodnota* `valid_length` *nesmí nikdy překročit hodnotu* `buffer_capacity`.“
- **Vysoká soudržnost (*High Cohesion*):**
  - Proměnné, které se mění společně a navzájem na sobě závisí, musí být seskupeny ve stejné struktuře.
  - Nikdy nepředáváme datový buffer a jeho délku jako dva nesouvisející argumenty, pokud tvoří logický celek.
- **Ochrana invariantu:**
  - V jazyce C nemáme privátní (`private`) proměnné jako v C++. Ochranu invariantu řešíme konvencí: uživatel nesmí modifikovat obsah struktury přímo, ale výhradně přes poskytnuté API funkce.

Praktická ukázka: Ochrana invariantu

Rozbití vs. dodržení vnitřní logiky struktury

```c
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

typedef struct {
    uint8_t *data;
    size_t active_length;
    size_t max_capacity;
} DataBuffer;

// API Function designed to strictly maintain the invariant:
// (active_length <= max_capacity)
bool buffer_append(DataBuffer *buf, uint8_t new_byte) {
    if (buf == nullptr || buf->active_length >= buf->max_capacity) {
        return false; // Invariant protected
    }
    buf->data[buf->active_length++] = new_byte;
    return true;
}

void application_logic(DataBuffer *my_buf) {
    // DANGEROUS: Direct modification breaks the invariant!
    // my_buf->active_length = 9999;
```

Abstraktní datové typy (ADT)

Absolutní skrytí implementačních detailů (Encapsulation)

- **Problém odhalených struktur:**
  - Pokud je definice struktury kompletně viditelná v hlavičkovém (`.h`) souboru, každý modul vidí její vnitřnosti.
  - Programátoři z lenosti začnou přistupovat k proměnným přímo, čímž vzniká těsná vazba (*Tight Coupling*). Změna vnitřku struktury pak rozbije celý projekt.

- **Řešení: Opaque Pointer (Neprůhledný ukazatel):**
  - Do hlavičkového souboru umístíme pouze dopřednou deklaraci struktury (*Forward Declaration*).
  - Skutečná definice struktury leží bezpečně ukryta ve zdrojovém (`.c`) souboru.

- **Výhody Opaque Pointeru v C:**
  - Volající kód má k dispozici pouze typizovaný ukazatel (tzv. *Handle*).
  - Zaručuje absolutní vynucení zapouzdření a striktní použití API. Jde o obdobu privátních tříd z C++ v čistém C (často označováno jako návrhový vzor PIMPL).

Praktická ukázka: Opaque Pointer (Hlavičkový soubor)

Definice veřejného rozhraní bez odhalení detailů

```c
// network_connection.h - Public API
#ifndef NETWORK_CONNECTION_H
#define NETWORK_CONNECTION_H

#include <stdbool.h>

// FORWARD DECLARATION (Opaque Pointer)
// The caller knows this type exists, but the compiler hides its size and members!
typedef struct NetworkConnection NetworkConnection;

// Public API
// Instantiation (Constructor equivalent)
NetworkConnection* net_connection_create(void);

// Behavior (Methods)
bool net_connection_connect(NetworkConnection *conn, const char *address);
void net_connection_send(NetworkConnection *conn, const char *msg);

// Destruction (Destructor equivalent)
void net_connection_destroy(NetworkConnection *conn);

#endif // NETWORK_CONNECTION_H
```

Samostatný úkol: Zapouzdřený PID regulátor (Context & Invarianty)

Časová dotace: 15–20 minut

Zadání úkolu:
Převeďte procedurální regulátor teploty využívající globální proměnné na znovupoužitelný modul využívající kontextový ukazatel (*Context Pointer*) a přísnou ochranu vnitřních invariantů.

Požadavky na implementaci:

1. Definujte kontextovou strukturu `PidController`, která sdružuje: koeficienty (`kp`, `ki`), akumulátor integrální složky (`integral_sum`) a limity výstupu (`out_min`, `out_max`).
2. Implementujte inicializační funkci `bool pid_init(...)`, která ověří vstupní invariant: `out\_min < out_max`.
3. Implementujte výpočetní funkci `float pid_compute(PidController *pid, float setpoint, float measurement)`, která během výpočtu striktně udrží vnitřní invariant (*anti-windup*: hodnota `integral_sum` ani výstup nesmí nikdy překročit mez `out_min` až `out_max`).
4. Vytvořte dvě nezávislé instance regulátoru (např. pro 2 topné zóny) a ověřte, že se jejich stavy vzájemně neovlivňují.

Modularita a návrh testovatelného firmware

Modularita v jazyce C

Fyzická struktura modulů a ochrana proti vícenásobnému vložení

- **Překladová jednotka (Translation Unit):**
  - Každý soubor `.c` je kompilátorem překládán samostatně do objektového souboru (`.o`).
  - Hlavičkový soubor (`.h`) definuje veřejné rozhraní (kontrakt) a exportované typy.
- **Include Guards vs.** `\#pragma once`**:**
  - **Klasické Include Guards**: `\#ifndef MODUL_H`, `\#define MODUL_H` ... Standardní dle ISO C, avšak hrozí kolize názvů v rozsáhlých projektech.
  - `\#pragma once`: Není formálně v normě ISO C, ale je de-facto podporován všemi moderními překladači (GCC, Clang, IAR, Keil, MSVC). Rychlejší zpracování preprocesorem a nulové riziko překlepu.
- **Zlaté pravidlo:** Hlavičkový soubor smí obsahovat deklarace, makra, typedefy a inline funkce – **nikdy** definice negenerických proměnných (jinak dojde k chybě linkeru: *multiple definition*).

Zapouzdření stavu modulu

Řízení viditelnosti pomocí `static` a `extern`

Klíčové slovo `static`:

- **Internal Linkage:** Symbol (funkce nebo globální proměnná) je viditelný **pouze** v rámci své překladové jednotky (`.c`).
- Linker tento symbol neexportuje do globální tabulky symbolů.
- Zabraňuje nechtěným zásahům z jiných modulů a kolizím jmen (obdoba `private` v C++).

Klíčové slovo `extern`:

- **External Linkage:** Deklaruje existenci globální proměnné nebo funkce definované v jiné překladové jednotce.
- V kvalitním embedded kódu se `extern` proměnným vyhýbáme – data mají být skryta za přístupovými funkcemi modulu.

Praktická ukázka: Zapouzdřený modul ovladače

Skrytý interní stav s bezpečným veřejným API

```c
// sensor_driver.c - Implementation file
#include <stdint.h>
#include <stdbool.h>

// Private module state: Invisible outside this translation unit
static uint32_t sample_counter = 0;
static bool is_calibrated = false;

// Private helper function: Inaccessible from outside
static int16_t apply_calibration_offset(int16_t raw_value) {
    return is_calibrated ? (raw_value - 12) : raw_value;
}

// Public API functions declared in sensor_driver.h
void sensor_driver_init(void) {
    sample_counter = 0;
    is_calibrated = true;
}

int16_t sensor_driver_read_value(void) {
    sample_counter++;
    int16_t raw = hardware_adc_read();
    return apply_calibration_offset(raw);
}
```

Architektura vrstev firmware (HAL)

Oddělení aplikační logiky od konkrétního křemíku

- **Problém monolitického kódu:**
  - Přímé volání hardwarových registrů (např. `GPIOA->ODR`) uvnitř stavového automatu aplikace znemožňuje testování na PC a přenos na jiný MCU.
- **Třívrstvá architektura firmware:**
  1. **Application Layer**: Stavové automaty, algoritmy, filtry, byznys logika. *Nesmí obsahovat žádné závislosti na čipu.*
  2. **HAL / Driver Interface**: Abstraktní rozhraní pro komunikaci (I2C, UART, SPI, GPIO).
  3. **BSP / Low-Level Drivers**: Konkrétní implementace pro daný mikrokontrolér (STM32, NXP, ESP32).
- **Cíl:** Aplikační kód lze beze změn zkompilovat na hostitelském PC s Linuxem či Windows a otestovat v řádu milisekund.

Typy knihoven

Od dynamického linkování po STB styl

- **Statické vs. Dynamické knihovny:**
  - **Statické (`.a`, `.lib`)**: Kód se fyzicky zkopíruje do výsledné binárky během linkování. Binárka je větší, ale nezávislá.
  - **Dynamické (`.so`, `.dll`)**: Kód se načítá až za běhu operačním systémem. Šetří RAM, pokud knihovnu sdílí více procesů, ale hrozí *DLL Hell* (problémy s verzemi).

- **Header-only knihovny:**
  - Moderní a velmi populární přístup (např. knihovny STB).
  - Veškerý kód (deklarace i implementace) leží v jednom `.h` souboru.
  - Uživatel v právě **jednom** zdrojovém souboru definuje speciální makro (např. `\#define STB_IMAGE_IMPLEMENTATION`), čímž aktivuje překlad implementace. V ostatních souborech slouží jen jako běžná hlavička.

Praktická ukázka: Header-only knihovna

Využití preprocesoru pro podmíněný překlad implementace

```c
// --- tiny_math.h (Header-only library) ---
#pragma once

// 1. PUBLIC API (Declarations available everywhere)
int tiny_add(int a, int b);

// 2. IMPLEMENTATION (Compiled ONLY where explicitly requested)
#ifdef TINY_MATH_IMPLEMENTATION

// This code is inserted into the translation unit only once
int tiny_add(int a, int b) {
    return a + b;
}

#endif // TINY_MATH_IMPLEMENTATION

// --- main.c ---
#define TINY_MATH_IMPLEMENTATION // Generates the actual binary code here
#include "tiny_math.h"
```

Dependency Injection (Vstřikování závislostí) v C

Předávání rozhraní pomocí struktur funkčních ukazatelů

Princip Dependency Injection

Modul vyšší úrovně si sám nevytváří ani napevno nevolá konkrétní hardwarový ovladač. Místo toho při inicializaci obdrží ukazatel na strukturu obsahující funkce rozhraní (*VTable / Driver Interface*).

V produkčním běhu (Cílový HW):

- Modulu předáme strukturu s funkcemi volajícími skutečné hardwarové registry MCU.

Při Unit testování (Host PC):

- Modulu předáme Mock strukturu, která zapisuje do paměťového pole a simuluje reakce hardware.

Praktická ukázka: Návrh rozhraní v C

Komunikační modul nezávislý na použitém hardware

```c
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

// Abstract hardware interface definition
typedef struct {
    bool (*send_byte)(uint8_t byte);
    bool (*receive_byte)(uint8_t *out_byte);
} UartInterface;

typedef struct {
    const UartInterface *io;
    uint32_t packets_sent;
} TelemetryService;

void telemetry_init(TelemetryService *service, const UartInterface *io_interface) {
    service->io = io_interface;
    service->packets_sent = 0;
}

bool telemetry_send_ping(TelemetryService *service) {
    if (service->io == nullptr || service->io->send_byte == nullptr) {
        return false;
    }
```

Samostatný úkol: Návrh HAL rozhraní a Dependency Injection

Časová dotace: 15–20 minut

Zadání úkolu:
Oddělte aplikační logiku teplotního alarmu od přímého volání hardwarových registrů pomocí struktury funkčních ukazatelů (*Dependency Injection*) a skryjte privátní počítadlo poplachů uvnitř překladové jednotky pomocí `static`.

Požadavky na implementaci:

1. Definujte abstraktní rozhraní `ThermalHalInterface` s ukazateli na funkce: `int16_t (*read_temp_celsius)(void)` a `void (*set_cooling_fan)(bool state)`.
2. Vytvořte aplikační strukturu `ThermalMonitor` uchovávající ukazatel `const ThermalHalInterface *hal` a prahovou teplotu `int16_t threshold_celsius`.
3. Definujte interní proměnnou modulu `static uint32_t total_alarms_triggered` a funkci `bool thermal_monitor_step(ThermalMonitor *mon)`, která přečte teplotu přes rozhraní, sepne ventilátor při překročení limitu a inkrementuje interní počítadlo.

Psaní testovatelného kódu a mockování

Strategie testování embedded systémů

Unit testování na hostitelském PC vs. na cílovém hardware

Testování na Host PC (x86-64 / ARM):

- **Extrémní rychlost:** Tisíce testů proběhnou za sekundy.
- **CI/CD automatizace:** Spouštění při každém `git push` na serveru bez nutnosti připojeného HW.
- **Pokročilé nástroje:** Běh pod AddressSanitizerem (ASan), Valgrindem a nástroji pro code coverage (gcov/lcov).

Testování na cílovém HW (HIL):

- **Fyzické ověření:** Skutečné elektrické úrovně, impedance sběrnic a napájení.
- **Hardwarové časování:** Přesná latence přerušení, DMA přenosy a chování hodinových zdrojů (PLL).
- **Nevýhoda:** Pomalé nahrávání přes JTAG/SWD, omezená škálovatelnost v týmu.

Zlaté pravidlo testovací pyramidy

Veškerou aplikační logiku, parsery paketů a stavové automaty testujeme na hostitelském PC s podvrženým HW. Cílový mikrokontrolér ověřuje pouze integraci periferií a časování.

Principy testovatelného kódu v C

Eliminace skrytých vazeb pro snadné mockování

- **1. Zákaz přímého volání registrů v logice:**
  - Pokud stavový automat přímo dereferencuje `*(uint32_t *)0x40020000UL`, kompilace na PC okamžitě zhavaruje na Segmentation Fault.
- **2. Čisté funkce (Pure Functions) a determinismus:**
  - Funkce, jejichž výstup závisí výhradně na vstupních parametrech (např. výpočet CRC, dekódování protokolu), jsou triviálně testovatelné.
- **3. Explicitní resetovatelnost stavu:**
  - Moduly se soukromým stavem (`static`) musí v testovacím režimu exportovat funkci pro reset do výchozího stavu mezi jednotlivými testy (`module_reset_for_test()`).
- **4. Techniky substituce závislostí:**
  - Rozhraní pomocí funkčních ukazatelů (Dependency Injection za běhu).
  - Nahrazení funkcí v době linkování (Link-time substitution přes GNU `-wrap`).

Mockování pomocí GNU Linker Wrap

Substituce hardwarových funkcí bez změny produkčního kódu

- GNU linker (`ld`) nabízí přepínač: `-Wl,-wrap=symbol`
  - Všechna volání funkce `symbol()` jsou automaticky přesměrována na funkci `__wrap_symbol()`.
  - Původní reálnou funkci lze stále zavolat přes `__real_symbol()`.

```c
// Production code calls hardware_i2c_write() directly:
// bool hardware_i2c_write(uint8_t dev_addr, const uint8_t *data, size_t len);

// In test_suite.c: Compile with -Wl,--wrap=hardware_i2c_write
#include <stdint.h>
#include <stdbool.h>

static uint8_t mock_last_byte_written = 0;
static bool mock_force_bus_error = false;

// Linker intercepts hardware_i2c_write and jumps here!
bool __wrap_hardware_i2c_write(uint8_t dev_addr, const uint8_t *data, size_t len) {
    (void)dev_addr;
    if (mock_force_bus_error) {
        return false; // Simulate hardware bus NACK failure
    }
    if (len > 0) {
        mock_last_byte_written = data[0];
    }
    return true;
```

Testovací framework Unity

Standard pro Unit testování v čistém C

- **Charakteristika Unity (ThrowTheSwitch.org):**
  - Extrémně lehký testovací framework napsaný v čistém C (pouhé 2 soubory: `unity.c` a `unity.h`).
  - Nevyžaduje dynamickou haldu ani C++ runtime. Lze spustit na PC i přímo na MCU přes sériovou linku!
- **Základní životní cyklus testu:**
  - `setUp()`: Inicializace spouštěná před každým jednotlivým testem.
  - `tearDown()`: Úklid spouštěný po každém testu.
- **Běžná assertovací makra v embedded:**
  - `TEST_ASSERT_TRUE(condition)`
  - `TEST_ASSERT_EQUAL_INT(expected, actual)`
  - `TEST_ASSERT_EQUAL_HEX8(expected, actual)`
  - `TEST_ASSERT_EQUAL_HEX8_ARRAY(expected, actual, length)`

Praktická ukázka: Unit test v Unity

Ověření chování protokolu s podvrženým mock rozhraním

```c
#include "unity.h"
#include <stdbool.h>
#include <stdint.h>

// Mock variables to inspect interaction
static uint8_t sent_payload[8];
static size_t sent_count = 0;

static bool mock_uart_send(uint8_t byte) {
    if (sent_count < sizeof(sent_payload)) {
        sent_payload[sent_count++] = byte;
        return true;
    }
    return false;
}

void setUp(void) {
    sent_count = 0; // Reset mock state before each test case
}

void tearDown(void) {}

void test_telemetry_ping_should_send_magic_byte(void) {
    UartInterface mock_io = { .send_byte = mock_uart_send, .receive_byte = nullptr };
```

Samostatný úkol: Unit testování se simulací chyby sběrnice

Časová dotace: 15–20 minut

Zadání úkolu:
Napište sadu unit testů na hostitelském PC ve stylu frameworku **Unity**, která pomocí podvrženého mocku (*Mock Object*) ověří chování ovladače baterie při výpadku komunikace na sběrnici I^2C (simulace NACK).

Požadavky na implementaci:

1. Mějme produkční funkci `bool battery_check_low_voltage(const I2cInterface *bus, bool *out_is_low)`, která přečte 16bitové napětí v mV z adresy `0x55` a nastaví `*out_is_low = true`, pokud je napětí < 3300 mV.
2. Vytvořte mock funkci `mock_i2c_read_u16`, kterou lze před každým testem nakonfigurovat pomocí globálních proměnných `mock_return_voltage` a `mock_force_nack_error`.
3. Napište funkci `setUp()` pro reset stavu mocku a 3 unit testy: (a) normální napětí 3800 mV, (b) nízké napětí 3100 mV, (c) selhání sběrnice (`mock_force_nack_error = true`).

Regresní testování a ladění chyb

Význam regresního testování v embedded C

Proč se opravené chyby v jazyce C tak snadno vracejí

- **Specifika jazyka C a skryté závislosti:**
  - Absence kontroly mezí u ukazatelů a polí.
  - Vedlejší efekty makro-expanzí, implicitních typových konverzí a paměťového aliasingu.
  - Oprava chyby na jednom místě může změnit paměťové rozvržení a nechtěně odmaskovat spící bug v jiném modulu.
- **Co je to regresní test:**
  - Automatizovaný test napsaný specificky pro ověření, že dříve odhalená a opravená chyba se znovu neobjevila.
  - Stává se trvalou součástí testovací sady spouštěné v CI/CD pipeline při každé změně kódu.
- **Cíl:** Zajištění, že oprava bugu je deterministická, nerozbíjí existující funkcionalitu a firmware dlouhodobě konverguje ke stabilitě.

TDD workflow při opravě chyb

Strukturovaný proces eliminace nahlášeného bugu

Čtyřkrokový cyklus opravy:

1. **Izolace a reprodukce:**
   - Extrakce minimálního vstupního vektoru způsobujícího selhání.
2. **Napsání selhávajícího testu (Red):**
   - Vytvoření nového testu v Unity simulujícího přesný scénář chyby. Test **musí** selhat!
3. **Oprava v kódu (Green):**
   - Minimální zásah do produkčního kódu, dokud test neprojde.
4. **Refaktoring a fixace:**
   - Zajištění čitelnosti a zařazení do CI sady.

Proč test psát PŘED opravou:

- Pokud test bez kódu opravy neselže, netestujete skutečnou příčinu problému!
- Zabraňuje klamnému pocitu bezpečí z falešně pozitivních testů.
- Slouží jako živá dokumentace hraničních stavů systému.

Praktická ukázka: Chyba v parseru zpráv

Off-by-one chyba při kontrole meze délky paketu

```c
// parser.c - Buggy production code
#include <stdint.h>
#include <stdbool.h>

#define MAX_PAYLOAD_LEN 8U

typedef struct {
    uint8_t payload[MAX_PAYLOAD_LEN];
    uint8_t length;
} Frame;

// BUG: Using '>' instead of '>=' allows length == 9, corrupting memory!
bool frame_parse_payload(Frame *frame, const uint8_t *raw_stream, uint8_t len) {
    if (len > (MAX_PAYLOAD_LEN + 1U)) {
        return false; // Faulty boundary check!
    }
    for (uint8_t i = 0; i < len; ++i) {
        frame->payload[i] = raw_stream[i]; // Buffer overflow if len == 9!
    }
    frame->length = len;
    return true;
}
```

Praktická ukázka: Regresní test a oprava

Izolace hraničního stavu v testovacím frameworku Unity

```c
// test_regression_parser.c - Unity test suite
#include "unity.h"

void test_regression_issue_402_payload_boundary_overflow(void) {
    Frame frame = {0};
    const uint8_t attack_vector[9] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

    // Must return false and reject packet exceeding MAX_PAYLOAD_LEN
    bool success = frame_parse_payload(&frame, attack_vector, sizeof(attack_vector));

    TEST_ASSERT_FALSE_MESSAGE(success, "Parser accepted payload larger than buffer!");
}

// parser.c - Patched safe implementation
bool frame_parse_payload_fixed(Frame *frame, const uint8_t *raw_stream, uint8_t len) {
    if (len > MAX_PAYLOAD_LEN) { // Correct strict boundary check
        return false;
    }
    for (uint8_t i = 0; i < len; ++i) {
        frame->payload[i] = raw_stream[i];
    }
    frame->length = len;
    return true;
}
```

Spouštění testů v CI/CD bez fyzického HW

Emulace ARM Cortex-M v prostředí QEMU

- **Omezení Unit testů na PC:**
  - Test na hostu (x86-64) neodhalí chyby specifické pro cílovou architekturu (např. zarovnání paměti, instrukční odlišnosti Cortex-M, inline assembler).
- **Řešení: Emulátor QEMU (`qemu-system-arm`):**
  - Emuluje kompletní instrukční sadu ARM Thumb-2 i periferní registry základních čipů (např. `lm3s6965evb` nebo STM32).
  - Umožňuje zkompilovat testovací firmware přímo cross-překladačem `arm-none-eabi-gcc` a spustit jej na virtuálním CPU.
- **ARM Semihosting:**
  - Mechanismus, kterým může firmware běžící v emulátoru nebo debuggeru posílat textový výstup do konzole hostitelského PC a předat návratový kód ukončení (`exit code`).

Praktická ukázka: Ukončení QEMU přes Semihosting

Předání návratového kódu z virtuálního MCU do CI/CD pipeline

```c
#include <stdint.h>

#define SEMIHOSTING_SYS_EXIT 0x18
#define ADP_STOPPED_APPLICATION_EXIT 0x20026

// Triggers ARM semihosting call via software breakpoint instruction
static inline void semihosting_exit(int32_t exit_code) {
    uint32_t parameters[2] = {
        ADP_STOPPED_APPLICATION_EXIT,
        (uint32_t)exit_code
    };

    register uint32_t r0 __asm__("r0") = SEMIHOSTING_SYS_EXIT;
    register uint32_t r1 __asm__("r1") = (uint32_t)parameters;

    // ARM Cortex-M semihosting trap instruction
    __asm__ volatile(
        "bkpt 0xAB"
        : : "r"(r0), "r"(r1) : "memory"
    );
}

void unity_suite_complete_handler(int failed_tests_count) {
    // 0 = Success (Pipeline Green), 1 = Failure (Pipeline Red)
```

Postmortální analýza a rekonstrukce pádu

Dekódování HardFault Handleru a přetavení crash logu v test

```
// Assembly stub passes stack pointer to C diagnostics handler
void HardFault_Handler(void) {
    __asm__ volatile(
        "tst lr, #4            \n" // Check EXC_RETURN bit 2 to determine active stack
        "ite eq                \n"
        "mrseq r0, msp         \n" // Argument 0 = Main Stack Pointer
        "mrsne r0, psp         \n" // Argument 0 = Process Stack Pointer
        "b hardfault_c_handler\n"
    );
}

void hardfault_c_handler(uint32_t *stack_frame) {
    // Extract hardware-saved registers from stack frame:
    uint32_t r0  = stack_frame[0];
    uint32_t pc  = stack_frame[6]; // Program Counter: Exact faulting instruction!
    uint32_t psr = stack_frame[7]; // Program Status Register

    // Diagnostic registers on Cortex-M3/M4/M7:
    volatile uint32_t cfsr = (*(volatile uint32_t *)0xE000ED28UL); // Fault Status
    (void)r0; (void)pc; (void)psr; (void)cfsr;
    while (1) { /* Halt CPU for SWD debugger inspection */ }
}
```

- Z hodnoty registru PC určíme přesné místo pádu v `.elf` souboru (`addr2line`) a následně

Samostatný úkol: TDD oprava celočíselného podtečení

Časová dotace: 15–20 minut

Zadání úkolu:
V produkčním dekodéru síťových paketů byla nahlášena kritická chyba: pokud útočník pošle v hlavičce paketu hodnotu `total_length = 2` (menší než 4bajtová hlavička), výpočet délky dat `uint16_t payload_len = total_length - 4U` podteče na hodnotu `65534` a způsobí pád systému (*HardFault*).

Požadavky na implementaci (TDD Red-Green cyklus):

1. Reprodukujte chybu napsáním regresního testu `test_regression_issue_512_header_underflow`, který zavolá parser se vstupem `total_length = 2`.
2. Ověřte, že test na původní chybné funkci selže (**Red**).
3. Opravte produkční funkci tak, aby před odečtením délky hlavičky zkontrolovala minimální přípustnou velikost rámce (**Green**).

Tvorba korektních programů a diagnostika

Nedefinované chování (Undefined Behavior)

Proč je UB v embedded systémech kritickým rizikem

Definice podle normy ISO C

Nedefinované chování je stav, kdy standard nestanoví žádné požadavky na chování překladače ani výsledného programu. Překladač smí generovat libovolný kód, vymazat celou větev nebo způsobit pád jádra.

- **Jak uvažuje moderní optimalizátor (GCC, Clang):**
  - Překladač předpokládá, že k UB **nikdy nemůže dojít**.
  - Pokud nějaká podmínka vede k UB, překladač ji vyhodnotí jako mrtvý kód (*dead code*) a zcela ji ze stroje odstraní!
- **Nejčastější zdroje UB v embedded C:**
  - Znaménkové celočíselné přetečení (*Signed Integer Overflow*).
  - Bitový posuv o hodnotu větší nebo rovnou šířce typu, posuv záporného čísla.
  - Dereference neplatného ukazatele, dereference `nullptr`.
  - Porušení pravidel zarovnání paměti (*Alignment Violation*).
  - Porušení pravidel aliasingu (*Strict Aliasing Rule*).

Praktická ukázka: Optimalizace na základě UB

Jak překladač odstraní kontrolu přetečení paměti

```c
#include <stdint.h>
#include <stdbool.h>

// DANGEROUS: Signed integer overflow is Undefined Behavior!
bool is_addition_safe_bad(int32_t current_val, int32_t delta) {
    // Programmer expects: if overflow wraps to negative, catch it
    // Reality (-O2/-O3): Compiler knows signed overflow is UB.
    // Therefore, current_val + delta CANNOT be less than current_val!
    // The entire 'if' statement is OPTIMIZED OUT of the binary!
    if (current_val + delta < current_val) {
        return false; // Never executed in generated assembly!
    }
    return true;
}

// CORRECT: Check boundary before performing the operation
bool is_addition_safe_correct(int32_t current_val, int32_t delta) {
    if (delta > 0 && current_val > (INT32_MAX - delta)) {
        return false; // Arithmetic bounds validated safely
    }
    return true;
}
```

Diagnostika: Sanitizery překladače

Odhalení paměťových a číselných chyb při běhu na PC

- Moderní překladače (GCC, Clang) obsahují instrumentační nástroje integrované přímo do kódu při překladu pro Host PC.

AddressSanitizer (ASan):

- Překlad: `-fsanitize=address`
- Odhaluje: Přetečení bufferu na haldě i zásobníku (*Buffer Overflow*), čtení uvolněné paměti (*Use-After-Free*), vícenásobné uvolnění (*Double Free*).
- Nahrazuje pomalý Valgrind.

UndefinedBehaviorSanitizer (UBSan):

- Překlad: `-fsanitize=undefined`
- Odhaluje: Znaménková přetečení, neplatné bitové posuvy, dereference nezarovnaných ukazatelů.
- Okamžitý záchyt chyb v runtime.

```
# Recommended GCC/Clang flags for compiling Unit Tests on Host PC:
# gcc -Wall -Wextra -Wpedantic -fsanitize=address,undefined -g test_suite.c
```

Statická analýza kódu

Hledání defektů bez nutnosti spouštění programu

- **1. Striktní varování překladače jako první linie obrany:**
  - Přepínače pro cross-kompilátor:
    `-Wall -Wextra -Wpedantic -Werror`
    `-Wconversion -Wshadow -Wstrict-prototypes -Wundef`
  - Pravidlo čistého sestavení: Každé varování překladače je považováno za chybu.
- **2. Dedikované analyzátory (Clang-Tidy, Cppcheck):**
  - **Cppcheck**: Specializovaný na jazyk C a embedded. Vynikající v odhalování neinicializovaných proměnných, dereferencí nulových ukazatelů a úniků prostředků s minimem falešně pozitivních hlášení.
  - **Clang-Tidy**: Kontrola adherence k certifikačním standardům (CERT C), kontrola pojmenovávání a odhalování zastaralých konstrukcí.

Standard MISRA C

Průmyslový standard pro spolehlivost a funkční bezpečnost

- **Původ a význam (Motor Industry Software Reliability Association):**
  - Vyvinuto pro automotive (ISO 26262), dnes de-facto standard pro letectví, medicínu (IEC 62304) i průmyslové řízení (IEC 61508).
  - Aktuální verze: **MISRA C:2012** (s dodatky Amendment 1–4) a **MISRA C:2023**.
- **Kategorizace pravidel:**
  - **Mandatory**: Zákaz výjimek. Kód musí pravidlo striktně splňovat (např. nepoužívat neinicializované proměnné).
  - **Required**: Pravidlo musí být splněno, nebo musí existovat formální písemná odchylka (*Deviation*) schválená architektem.
  - **Advisory**: Doporučená dobrá praxe.

Klíčová pravidla MISRA C v praxi

Příklady striktních omezení jazyka C

- **Pravidlo 4.6 (Typová čistota):** Zákaz základních typů `int`, `short`, `long`. Povinnost používat striktní typy z `<stdint.h>`.
- **Pravidlo 11.4 / 11.8 (Konverze ukazatelů):** Zákaz svévolného přetypování mezi ukazateli různých typů nebo odstranění `const`.
- **Pravidlo 21.3 (Dynamická paměť):** Zákaz funkcí `malloc`, `calloc`, `realloc` a `free`.

```c
#include <stdint.h>

// MISRA VIOLATION: Using implicit int, missing const, and unparenthesized macro
#define BUFFER_LIMIT 100 + 20
int process_data(int *ptr);

// MISRA COMPLIANT: Explicit width, explicit const, protected macro expression
#define BUFFER_LIMIT_SAFE ((uint32_t)(100U + 20U))

int32_t process_data_safe(const uint8_t *payload, uint32_t length);
```

Samostatný úkol: Refaktoring kódu dle MISRA C a eliminace UB

Časová dotace: 15–20 minut

Zadání úkolu:
Proveďte bezpečnostní audit a refaktoring funkce pro výpočet kalibrovaného tlaku tak, aby neobsahovala žádné **Undefined Behavior (UB)** a splňovala pravidla standardu **MISRA C**.

Chybný kód k přepracování:

```
#define SCALE_OFFSET 10 + 2
int calibrate_sensor(int *raw_ptr, int shift) {
    int val = *raw_ptr * SCALE_OFFSET; // Macro precedence bug & UB overflow!
    return val << shift;               // UB if shift < 0 or shift >= 32!
}
```

Požadavky na opravu:

1. Opravte makro `SCALE_OFFSET` řádným uzávorkováním a typovým sufixem `U` (MISRA C).
2. Nahraďte základní typ `int` explicitními typy z `<stdint.h>` a přidejte kvalifikátor `const` k vstupnímu ukazateli (MISRA Pravidla 4.6 a 11.8).
3. Ošetřete nulový ukazatel, přetečení při násobení/sčítání před samotnou operací a platný rozsah bitového posuvu (`shift < 32U`).

Polymorfismus v jazyce C

Simulace dědičnosti v C

Podstruktura jako první položka

- **Absence nativních objektů:**
  - Jazyk C nemá klíčová slova `class`, `extends` ani virtuální tabulky metod (vtable).
  - Přesto můžeme objektový návrh a polymorfismus snadno simulovat pomocí struktur a ukazatelů na funkce.
- **Princip vkládání (Embedding):**
  - Základní (rodičovskou) strukturu vložíme jako **úplně první položku** do odvozené (dceřiné) struktury.
  - Standard jazyka C garantuje, že ukazatel na strukturu vždy ukazuje přesně na její první položku (neexistuje žádný padding na offsetu 0).
- **Polymorfní volání:**
  - Díky nulovému offsetu můžeme bezpečně přetypovat ukazatel na dceřinou strukturu na ukazatel na rodičovskou strukturu a předat jej obecné funkci.

Praktická ukázka: Simulace dědičnosti

Přetypování dceřiné struktury na rodičovskou

```c
#include <stdint.h>
#include <stdio.h>

// "Base class" representing a generic sensor
typedef struct {
    uint32_t id;
    void (*read_value)(void); // Virtual method
} SensorBase;

// "Derived class" representing a specific temperature sensor
typedef struct {
    SensorBase base; // MUST be the very first member!
    float current_temperature;
} TemperatureSensor;

// Polymorphic function accepting ANY derived sensor
void process_sensor(SensorBase *sensor) {
    printf("Processing sensor ID: %u\n", sensor->id);
    if (sensor->read_value != nullptr) {
        sensor->read_value(); // Dynamic dispatch
    }
}
```

Makro `container_of` a `offsetof`

Jak získat rodiče, když není na začátku struktury

- **Problém s vícenásobným děděním:**
  - Pokud dceřiná struktura obsahuje více bázových tříd nebo uzlů, pouze jedna z nich může být na offsetu 0.
  - Máme-li ukazatel na vnitřní prvek, jak zjistíme adresu celé nadřazené struktury?
- **Makro `offsetof`:**
  - Definováno v `<stddef.h>`. Vrací posun (v bajtech) konkrétní položky od začátku struktury.
- **Idiom `container_of`:**
  - Zpopularizováno v jádře OS Linux.
  - Pokud známe adresu členské proměnné, její jméno a typ nadřazené struktury, odečteme `offsetof` od adresy člena. Tím získáme přesný začátek (adresu) obalující (kontejnerové) struktury.



Praktická ukázka: Implementace container_of

Výpočet adresy kontejneru pomocí ukazatelové aritmetiky

```c
#include <stddef.h>
#include <stdint.h>

// Standard Linux kernel macro translated for generic C
#define CONTAINER_OF(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))

typedef struct {
    uint32_t timestamp;
    uint32_t hardware_id; // Let's say we only have a pointer to THIS member
    float value;
} ComplexSensor;

void process_hardware_id(uint32_t *hw_id_ptr) {
    // We only have the pointer to 'hardware_id', but we need the whole struct!
    // By subtracting the offset, we safely recover the parent structure pointer.
    ComplexSensor *parent = CONTAINER_OF(hw_id_ptr, ComplexSensor, hardware_id);

    // Now we can access siblings
    parent->timestamp = 0;
}
```

Intruzivní datové struktury

Kombinace polymorfismu a efektivní správy paměti

- **Standardní (neintruzivní) seznam:**
  - Uzel seznamu obsahuje ukazatel `next` a ukazatel `void *data` (nebo data přímo kopíruje).
  - Vyžaduje dynamickou alokaci paměti (`malloc`) pro každý nový uzel seznamu. V embedded světě nevhodné.
- **Intruzivní (*Intrusive*) seznam:**
  - Datová struktura (např. náš senzor) sama v sobě obsahuje uzel seznamu (např. `ListNode node;`).
  - Spojový seznam propojuje tyto vnitřní uzly, nikoliv samotná data.
  - Jakmile algoritmus iteruje přes uzly, pomocí makra `container_of` získá zpět původní uživatelská data.
- **Přínos pro embedded (RTOS):**
  - Absolutně nulová dynamická alokace, 100% determinismus a žádná externí fragmentace paměti.

Praktická ukázka: Intruzivní spojový seznam

Použití vnitřního uzlu a vytažení obalových dat

```c
#include <stddef.h>

#define CONTAINER_OF(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))

// Generic list node (Does NOT know about the payload!)
typedef struct ListNode {
    struct ListNode *next;
} ListNode;

// Application specific data payload containing the node
typedef struct {
    float voltage;
    ListNode list_element; // Embedded intrusive node
} PowerMonitor;

void iterate_list(ListNode *head) {
    ListNode *current = head;
    while (current != nullptr) {
        // Recover the PowerMonitor object from the internal list node
        PowerMonitor *monitor = CONTAINER_OF(current, PowerMonitor, list_element);

        // Process the payload...
        current = current->next;
```

Samostatný úkol: Intruzivní fronta časovačů (container_of)

Časová dotace: 15–20 minut

Zadání úkolu:
Implementujte správce softwarových časovačů využívající intruzivní spojový seznam a makro `CONTAINER_OF` pro bezpečné získání nadřazené struktury bez dynamické alokace paměti.

Požadavky na implementaci:

1. Definujte obecný uzel intruzivního seznamu `ListNode` (obsahující pouze ukazatel `next`) a makro `CONTAINER_OF(ptr, type, member)` využívající `offsetof`.
2. Definujte aplikační strukturu `SoftTimer`, která obsahuje: `uint32_t period_ms`, `const char *name` a vnitřní uzel `ListNode node` (umístěný úmyslně až jako poslední položku struktury).
3. Implementujte funkci `void timers_tick_all(ListNode *head)`, která projde seznam uzlů, pomocí `CONTAINER_OF` zrekonstruuje ukazatel na `SoftTimer` a vypíše jeho název a periodu.

Výčtový polymorfismus

Identifikace typu za běhu

Použití `enum` jako značky (Type Tag)

- **Dynamické chování v C:**
  - Pokud pracujeme s kolekcí různých objektů (např. pole zpráv, strom syntaktické analýzy), potřebujeme za běhu poznat, o jaký typ dat se jedná.
  - Jazyk C nemá RTTI (*Run-Time Type Information*) jako C++.
- **Výčtový typ (`enum`):**
  - Představuje ideální a paměťově efektivní nástroj pro vytvoření „značky“ (*Tag*).
  - Typicky zabírá 1 až 4 bajty (záleží na kompilátoru a rozsahu hodnot, v C23 lze specifikovat podkladový typ, např. `enum : uint8_t`).
- **Rozhodovací logika:**
  - Zpracování probíhá pomocí konstrukce `switch`. Kompilátor u `enum` dokáže varovat, pokud programátor zapomene pokrýt některou z možností (warning `-Wswitch`).

Praktická ukázka: enum jako identifikátor

Struktura nesoucí informaci o svém vlastním typu

```c
#include <stdint.h>
#include <stdio.h>

// The "Tag" defining what kind of event we are dealing with
typedef enum : uint8_t {
    EVENT_KEYPRESS,
    EVENT_MOUSE_CLICK,
    EVENT_SYSTEM_SHUTDOWN
} EventType;

// A generic structure representing an event
typedef struct {
    EventType type;  // Identifier
    uint32_t  timestamp;
    // ... payload data would go here ...
} SystemEvent;

void process_event(SystemEvent *event) {
    if (event == nullptr) return;

    // The compiler ensures all enum values are handled in the switch
    switch (event->type) {
        case EVENT_KEYPRESS:
            printf("Processing keypress.\n");
```

Paměťová optimalizace

Alokace místa pomocí `union`

- **Problém s plýtváním pamětí:**
  - Pokud objekt může nabývat několika různých podob (např. zpráva může obsahovat buď souřadnice myši, NEBO kód klávesy), vložení obou datových sad do `struct` by znamenalo zbytečně velkou spotřebu paměti.
- **Konstrukce `union`:**
  - Na rozdíl od `struct`, kde se položky řadí za sebe, sdílejí všechny členy `union` **stejný paměťový prostor** (začínají na offsetu 0).
  - Velikost `union` je rovna velikosti jejího největšího člena (plus případný padding kvůli zarovnání).
- **Pravidla a rizika:**
  - V jeden okamžik je „aktivní“ pouze jeden člen.
  - Zapsání dat do člena A a následné čtení z člena B (tzv. *Type Punning*) bylo historicky nedefinované chování, ale moderní C standardy jej za určitých podmínek povolují (na rozdíl od C++).

Praktická ukázka: Překryv v paměti

Rozdíl ve velikosti mezi `struct` a `union`

```c
#include <stdint.h>
#include <stdio.h>

// A struct holds ALL members simultaneously. Size: 4 + 8 = 12 bytes (or 16 aligned)
typedef struct {
    uint32_t ip_address_v4;
    uint64_t mac_address;
} NetworkAddressStruct;

// A union holds ONLY ONE member at a time. Size: max(4, 8) = 8 bytes total
typedef union {
    uint32_t ip_address_v4;
    uint64_t mac_address;
} NetworkAddressUnion;

void demonstrate_union(void) {
    NetworkAddressUnion addr;

    // Writing to one member overwrites the shared memory space
    addr.ip_address_v4 = 0xC0A80101; // 192.168.1.1

    // If we write to the MAC address, the IP is lost
    addr.mac_address = 0x0011223344556677;
```

Tagged Union (Variant)

Imitace algebraického datového typu (ADT)

- **Spojení `enum` a `union`:**
  - Samotný `union` je nebezpečný, protože „neví“, který z jeho členů je právě aktivní.
  - Řešením je obalit `union` do struktury a přidat k němu `enum` tag.
- **Tzv. Tagged Union (označkovaná unie):**
  - Architektonický vzor, který umožňuje proměnné bezpečně nabývat různých datových typů a struktur.
  - Tento přístup nativně používá jazyk Rust (`enum`) nebo standardní knihovna C++ (`std::variant`).
- **Anonymní unie (C11):**
  - Od standardu C11 můžeme uvnitř struktury definovat `union` bez jména. K jejím členům pak lze přistupovat přímo, čímž kód získá na čistotě.

Praktická ukázka: Zápis Tagged Union v C

Základ pro bezpečné a paměťově efektivní polymorfní zprávy

```c
#include <stdint.h>
#include <stdio.h>

// 1. Define the tags
typedef enum : uint8_t {
    PAYLOAD_INT,
    PAYLOAD_FLOAT,
    PAYLOAD_COORD
} PayloadType;

// 2. Combine tag (enum) and payload (union) into a single struct
typedef struct {
    PayloadType type;      // The Tag tells us what is active

    union {                // Anonymous union (C11 feature)
        int32_t  i_val;
        float    f_val;
        struct {           // Nested struct inside the union
            int16_t x;
            int16_t y;
        } coord;
    };                     // No instance name required!
} VariantMessage;
```

Samostatný úkol: Zpracování heterogenních příkazů (Tagged Union)

Časová dotace: 15–20 minut

Zadání úkolu:
Navrhněte paměťově úspornou strukturu příkazu pro průmyslový aktuátor pomocí vzoru **Tagged Union** (kombinace C23 typovaného `enum` a anonymní `union`).

Požadavky na implementaci:

1. Definujte výčtový typ s explicitním podkladovým typem `typedef enum : uint8_t \{ CMD_MOVE, CMD_SET_TEMP, CMD_EMERGENCY_STOP \} CommandType;`.
2. Definujte strukturu `ActuatorCommand`, která obsahuje tag `CommandType type` a anonymní `union` se třemi variantami dat: souřadnice `struct \{ int16_t x; int16_t y; \} move`, cílová teplota `float target_temp` a chybový kód `uint32_t stop_reason`.
3. Implementujte funkci `void execute_command(const ActuatorCommand *cmd)` využívající konstrukci `switch` pro bezpečné zpracování všech tří variant a vypište celkovou velikost struktury pomocí `sizeof`.

Rekurzivní volání

Definice a druhy rekurze

Jak funkce volají samy sebe

- **Co je to rekurze:**
  - Programovací technika, při které funkce v rámci svého těla volá sama sebe, dokud není splněna definovaná podmínka.
  - Slouží k elegantnímu řešení problémů, které lze rozložit na menší instance téhož problému (např. procházení stromů, algoritmus *Divide and Conquer*).
- **Přímá rekurze (*Direct Recursion*):**
  - Funkce `A()` obsahuje přímo ve svém těle volání funkce `A()`.
  - Je snadno čitelná a kompilátor ji dokáže snadno analyzovat.
- **Nepřímá rekurze (*Indirect Recursion*):**
  - Funkce `A()` volá funkci `B()`, která následně volá zpět funkci `A()` (nebo přes delší řetězec funkcí).
  - V praxi je často hůře čitelná, hůře se ladí a ztěžuje statickou analýzu kódu.

Praktická ukázka: Druhy rekurze v kódu

Jak vypadá přímé a nepřímé zanoření

```c
#include <stdint.h>
#include <stdio.h>

// 1. Direct Recursion
void countdown_direct(uint32_t value) {
    if (value == 0) return;
    printf("Direct: %u\n", value);
    countdown_direct(value - 1); // Calls itself directly
}

// 2. Indirect Recursion (Forward declaration needed)
void process_even(uint32_t value);

void process_odd(uint32_t value) {
    if (value == 0) return;
    printf("Odd: %u\n", value);
    process_even(value - 1); // Calls counterpart
}

void process_even(uint32_t value) {
    if (value == 0) return;
    printf("Even: %u\n", value);
    process_odd(value - 1); // Calls counterpart, completing the cycle
}
```

Ukončovací podmínka (*Base Case*)

Prevence nekonečného cyklu a vyčerpání paměti

- **Kritický význam Base Case:**
  - Každá rekurzivní funkce **musí** obsahovat cestu, která nevede k dalšímu rekurzivnímu volání.
  - Bez této podmínky se rekurze stává nekonečnou (obdoba `while(1)`).
- **Spotřeba paměti (Stack Frames):**
  - Každé zanoření funkce vytvoří na zásobníku (*Stack*) nový rámec obsahující: návratovou adresu, uložené registry procesoru a lokální proměnné.
- **Embedded kontext a Stack Overflow:**
  - Mikrokontroléry mají zásobník v řádu jednotek kilobajtů. Hluboká rekurze zásobník rychle vyčerpá, což vede k tichému přepsání dat (pád do *HardFault*).
  - Z tohoto důvodu standard **MISRA C (Pravidlo 17.2)** zakazuje použití jakékoliv (přímé i nepřímé) rekurze v safety-critical systémech.

Praktická ukázka: Base case a spotřeba zásobníku

Výpočet faktoriálu a anatomie zanoření

```c
#include <stdint.h>

// Calculates factorial: n! = n * (n-1)!
uint32_t factorial(uint32_t n) {
    // BASE CASE: The recursion stops here.
    // Without this, 'n' would eventually underflow and recurse infinitely.
    if (n <= 1) {
        return 1;
    }

    // RECURSIVE CASE:
    // The CPU must save the current state, evaluate factorial(n - 1),
    // and then multiply the returned result by 'n'.
    // If n = 10, there will be 10 simultaneous stack frames in memory!
    return n * factorial(n - 1);
}
```

Optimalizace rekurze (Tail Call Optimization)

Jak kompilátor mění rekurzi na iteraci

- **Koncová rekurze (*Tail Recursion*):**
  - Zvláštní případ rekurze, kdy je rekurzivní volání **absolutně poslední** operací, která se ve funkci provede.
  - Po návratu z rekurzivního volání se už neprovádí žádná další kalkulace (např. chybí zpětné násobení jako u běžného faktoriálu).

- **Tail Call Optimization (TCO):**
  - Pokud kompilátor (GCC, Clang) detekuje koncovou rekurzi a je zapnuta optimalizace (`-O2`, `-O3`), provede optimalizaci volání.
  - Namísto vytváření nového rámce na zásobníku (*Stack Frame*) kompilátor pouze aktualizuje parametry a provede skok (`JMP`) na začátek funkce.
  - **Výsledek**: Rekurze se v binárním kódu změní na obyčejný paměťově bezpečný cyklus (spotřeba zásobníku je konstantní, $O(1)$).

Praktická ukázka: Tail Call Optimization

Transformace faktoriálu na koncovou rekurzi (s akumulátorem)

```c
#include <stdint.h>

// TAIL-RECURSIVE FACTORIAL
// 'accumulator' carries the ongoing result through the recursive calls.
uint32_t factorial_tail(uint32_t n, uint32_t accumulator) {
    if (n <= 1) {
        return accumulator; // BASE CASE
    }

    // TAIL CALL: The recursion is the very last operation.
    // There is no pending multiplication to do after the call returns.
    // An optimizing compiler will turn this into a loop, reusing the stack frame.
    return factorial_tail(n - 1, n * accumulator);
}

// Wrapper function to hide the accumulator from the user API
uint32_t calculate_factorial(uint32_t n) {
    return factorial_tail(n, 1);
}
```

Samostatný úkol: Koncová rekurze (TCO) pro výpočet CRC/mocniny

Časová dotace: 15–20 minut

Zadání úkolu:
Převeďte klasickou rekurzivní funkci pro výpočet celočíselné mocniny x^n na koncovou rekurzi (*Tail Recursion*) s akumulátorem a přidejte bezpečnostní limit hloubky zanoření pro ochranu zásobníku (*Stack Overflow* prevence).

Požadavky na implementaci:

1. Implementujte pomocnou funkci s koncovou rekurzí `uint64_t power_tail(uint32_t base, uint32_t exp, uint64_t acc)`, kde je rekurzivní volání absolutně poslední operací funkce.
2. Obalte ji veřejnou funkcí `bool safe_power(uint32_t base, uint32_t exp, uint64_t *out_res)`, která nejprve zkontroluje maximální povolenou hloubku (`exp <= 32U`), aby i bez zapnuté optimalizace nedošlo k přetečení zásobníku MCU.

Možnosti generického programování

Klíčové slovo `_Generic`

Typově závislé makro a simulace přetěžování funkcí

- **Přetěžování funkcí v C:**
  - Jazyk C (na rozdíl od C++) nativně nepodporuje přetěžování funkcí (*Function Overloading*) – nemůžeme mít více funkcí se stejným jménem, ale různými parametry.
  - Historicky se to obcházelo předponami typu `abs()`, `fabs()`, `labs()`.
- **Revoluce v C11 (`_Generic`):**
  - Klíčové slovo `_Generic` umožňuje makru provést rozhodnutí na základě **datového typu** předaného argumentu již během kompilace.
  - Funguje to podobně jako `switch`, ale místo hodnot se porovnávají typy.
- **Využití v praxi:**
  - Vytváření jednotného, typově bezpečného rozhraní API (např. jednotné makro `print(x)`).
  - Preprocesor vybere správnou specifickou funkci a na ni volání přesměruje.

Praktická ukázka: _Generic

Jednotné rozhraní pro různé datové typy

```c
#include <stdio.h>
#include <stdint.h>

// Specific backend functions
void print_int(int x) { printf("Int: %d\n", x); }
void print_float(float x) { printf("Float: %f\n", x); }
void print_string(const char *x) { printf("String: %s\n", x); }

// Generic macro front-end
#define print_data(X) _Generic((X), \
    int: print_int,                 \
    float: print_float,             \
    char*: print_string,            \
    const char*: print_string,      \
    default: print_int              \
)(X)

void application_logic(void) {
    print_data(42);          // Calls print_int(42)
    print_data(3.14f);       // Calls print_float(3.14f)
    print_data("Hello!");    // Calls print_string("Hello!")
}
```

Generování kódu pomocí maker

Metaprogramování v jazyce C

- **Potřeba generických struktur:**
  - V C nemáme k dispozici šablony (*Templates*) jako v C++. Pokud chceme typově bezpečný spojový seznam pro `int` i pro `float`, musíme obvykle kód zkopírovat a upravit.
- **Makra pro generování kódu (*X-Macros* / *Token Pasting*):**
  - Preprocesor C umožňuje pomocí operátoru `\#\#` spojovat textové tokeny.
  - Tím lze dynamicky generovat názvy struktur a funkcí (např. `Vector_int`, `Vector_float`).
- **Výhody a nevýhody:**
  - **Výhoda**: Zaručuje 100% typovou bezpečnost bez nutnosti přetypovávání ukazatelů `void*`. Žádná režie za běhu.
  - **Nevýhoda**: Vygenerovaný kód není v textovém editoru vidět, chyby překladače odkazují na vnitřek složitého makra a špatně se krokuje v debuggeru.

Praktická ukázka: Generátor vektorů

Makro, které definuje strukturu a API pro konkrétní typ

```
#include <stddef.h>

// Macro to generate a type-safe generic Vector struct and init function
#define DEFINE_VECTOR_TYPE(TYPE)                        \
    typedef struct {                                    \
        TYPE *data;                                     \
        size_t size;                                    \
        size_t capacity;                                \
    } Vector_##TYPE;                                    \
                                                        \
    void vector_init_##TYPE(Vector_##TYPE *vec) {       \
        if (vec != nullptr) {                           \
            vec->data = nullptr;                        \
            vec->size = 0;                              \
            vec->capacity = 0;                          \
        }                                               \
    }

// Instantiate the templates for specific types
DEFINE_VECTOR_TYPE(int)
DEFINE_VECTOR_TYPE(float)

void test_vectors(void) {
    Vector_int my_ints;
```

Vymazání typu (Type Erasure) a zarovnání

Univerzální práce s pamětí pomocí `void*`

- **Vymazání typu pomocí `void*`:**
  - Alternativa k makrům. Algoritmus nezná konkrétní typ, dostává pouze surovou adresu (`void*`) a velikost objektu v bajtech pomocí `sizeof`.
  - Typické využití: knihovní funkce `qsort()` a `bsearch()`.
- **Problém paměťového zarovnání (*Alignment*):**
  - Pokud alokujeme univerzální buffer bajtů (`uint8_t buffer[256]`), nemusí být na paměti zarovnán správně pro uložení např. 64bitového `double`.
  - Čtení 64bitového čísla z nezarovnané adresy vyvolá na ARM procesoru výjimku *HardFault*.
- **Nové vlastnosti v C11/C23 (`alignas`, `alignof`):**
  - `alignas(T)` vynutí zarovnání proměnné na hraniční adresu, která je kompatibilní s typem `T`.
  - `alignof(T)` (v C23 jako makro pro `_Alignof`) vrací, kolik bajtů zarovnání typ `T` vyžaduje.

Praktická ukázka: Bezpečný typově vymazaný buffer

Spojení `sizeof` a `alignas` pro univerzální úložiště

```c
#include <stdint.h>
#include <stddef.h>
#include <stdalign.h>

// A generic byte buffer that guarantees strict alignment
// suitable for the largest basic types (e.g., double, uint64_t).
// In C23, alignas is a standard keyword.
alignas(max_align_t) uint8_t generic_storage[1024];

// Type-erased function (works with ANY data type)
void generic_swap(void *a, void *b, size_t element_size) {
    uint8_t *byte_a = (uint8_t *)a;
    uint8_t *byte_b = (uint8_t *)b;

    // Swap byte by byte without knowing what the actual type is
    for (size_t i = 0; i < element_size; ++i) {
        uint8_t temp = byte_a[i];
        byte_a[i] = byte_b[i];
        byte_b[i] = temp;
    }
}
```

Samostatný úkol: Generické makro `_Generic` pro saturaci hodnot

Časová dotace: 15–20 minut

Zadání úkolu:
Vytvořte typově bezpečné generické rozhraní `clamp_val(val, min, max)` pomocí klíčového slova `_Generic` a generátoru kódu v preprocesoru (`\#\#`).

Požadavky na implementaci:

1. Pomocí makra `DEFINE_CLAMP_FUNC(SUFFIX, TYPE)` vygenerujte typově specifické funkce `clamp_i32`, `clamp_u16` a `clamp_f32` pomocí operátoru spojování tokenů `\#\#`.
2. Vytvořte zastřešující makro `clamp_val(X, MIN, MAX)` využívající `_Generic((X), ...)`, které během kompilace automaticky vybere správnou funkci podle typu prvního argumentu.

Objektově orientované programování

Identifikace typu za běhu (RTTI)

Jak nahradit chybějící RTTI v jazyce C

- **Absence nativní podpory:**
  - Jazyk C (na rozdíl od C++) nemá nativní *Run-Time Type Information* (RTTI). Neexistuje zde operátor jako `dynamic_cast` nebo `typeid`.
- **Implementace pomocí značek (Type Tags):**
  - Nejběžnějším způsobem, jak si „objekt“ pamatuje svůj skutečný typ, je vložení identifikátoru (typicky `enum`) přímo do bázové (základní) struktury.
  - Každá funkce, která přijímá obecný ukazatel na bázový typ, si může tento identifikátor přečíst a bezpečně přetypovat ukazatel na jeho skutečnou odvozenou strukturu.

Praktická ukázka: Type Tag v základní třídě

Uchování informace o typu pro bezpečné přetypování za běhu

```c
#include <stdint.h>
#include <stdio.h>

// 1. Enum representing all possible derived classes
typedef enum {
    CLASS_TYPE_BASE,
    CLASS_TYPE_CAR,
    CLASS_TYPE_TRUCK
} ClassType;

// 2. Base structure containing the type information
typedef struct {
    ClassType type_id;
} VehicleBase;

void identify_vehicle(VehicleBase *v) {
    if (v == nullptr) return;

    if (v->type_id == CLASS_TYPE_CAR) {
        printf("This is a Car.\n");
    } else if (v->type_id == CLASS_TYPE_TRUCK) {
        printf("This is a Truck.\n");
    }
}
```

Virtuální tabulky metod (VMT)

Dynamický polymorfismus pomocí ukazatelů na funkce

- **Ukazatele na funkce (Function Pointers):**
  - Proměnná, která neuchovává data, ale paměťovou adresu spustitelného kódu funkce. Umožňuje volání různých funkcí dynamicky za běhu.
- **Seskupení do VMT (Virtual Method Table):**
  - Vkládat ukazatele na funkce přímo do datové struktury každého objektu by znamenalo obrovské plýtvání pamětí.
  - Místo toho se vytvoří samostatná statická struktura obsahující **pouze** tyto ukazatele na metody (operace).
  - Každý objekt (instance) pak obsahuje pouze **jeden ukazatel** (`vptr`) odkazující na tuto sdílenou „tabulku virtuálních metod“.

Praktická ukázka: Definice VMT

Struktura sdružující operace nad objektem

```c
#include <stdint.h>
#include <stdio.h>

// Forward declaration of the context structure
typedef struct Shape Shape;

// 1. The Virtual Method Table (VMT) containing function pointers
typedef struct {
    void (*draw)(Shape *self);
    float (*calculate_area)(Shape *self);
} ShapeVTable;

// 2. The Base Class containing a pointer to its VMT (vptr)
struct Shape {
    const ShapeVTable *vptr; // Pointer to the shared operations table
    uint32_t color;
};

// Generic dispatch function mimicking OOP method call
void shape_draw(Shape *s) {
    if (s != nullptr && s->vptr != nullptr && s->vptr->draw != nullptr) {
        s->vptr->draw(s); // Dynamic dispatch
    }
}
```

Imitace dědičnosti a přepisování metod

Jak v C implementovat *Override* a *Inheritance*

- **Imitace dědičnosti (*Inheritance*):**
  - Odvozená (dceřiná) struktura vždy obsahuje bázovou (rodičovskou) strukturu jako svůj **první prvek**. To zaručuje nulový paměťový offset a bezpečné přetypování zpět na bázovou strukturu.
- **Výchozí operace (*Default Methods*):**
  - Bázová třída má svou vlastní VMT s generickými implementacemi (např. funkce, která jen vrátí chybu nebo nic neprovede).
- **Přepisování metod (*Override*):**
  - Odvozená třída definuje svou specifickou VMT. Do ní vloží ukazatele na své specifické implementace funkcí.
  - Pokud některou metodu odvozená třída nepřepíše, jednoduše do své VMT zkopíruje odkaz na původní výchozí metodu z bázové třídy.

Praktická ukázka: Přepsání metod (Override)

Vytvoření specifické VMT pro odvozený typ

```c
#include <stdio.h>

// (Assuming Shape and ShapeVTable are defined as in previous slide)
typedef struct Shape Shape;
typedef struct { void (*draw)(Shape*); } ShapeVTable;
struct Shape { const ShapeVTable *vptr; };

// Derived class (Inheritance imitation)
typedef struct {
    Shape base; // MUST be the very first member
    float radius;
} Circle;

// Circle's specific implementation of the draw method (Override)
void circle_draw_impl(Shape *self) {
    Circle *c = (Circle *)self; // Safe downcast
    printf("Drawing a circle with radius %.2f\n", c->radius);
}

// Global static VTable instance specifically for all Circles
const ShapeVTable CIRCLE_VTABLE = {
    .draw = circle_draw_impl
};
```

Samostatný úkol: Polymorfní komunikační kanál (VMT & Override)

Časová dotace: 15–20 minut

Zadání úkolu:
Implementujte tabulku virtuálních metod (**VMT**) pro abstraktní komunikační rozhraní `Channel` a vytvořte odvozenou třídu `EncryptedChannel`, která přepíše (*Override*) metodu pro odeslání bajtu.

Požadavky na implementaci:

1. Definujte bázovou strukturu `Channel` s ukazatelem `const ChannelVTable *vptr` a metodou `void (*send)(Channel *self, uint8_t byte)` ve VMT.
2. Definujte odvozenou strukturu `EncryptedChannel`, která má `Channel base` jako svůj **první člen** a navíc položku `uint8_t xor_key`.
3. Vytvořte specifickou VMT pro `EncryptedChannel`, která před výpisem bajt zašifruje operací `byte \textasciicircum {} xor_key`, a otestujte polymorfní volání přes ukazatel `Channel*`.

Bonus: Externí knihovny

SDL3 (Simple DirectMedia Layer)

Multimédia, okna a hardware na dosah ruky

- **Co je to SDL3:**
  - Nízkoúrovňová multiplatformní knihovna (Windows, macOS, Linux, iOS, Android) napsaná v C.
  - Poskytuje jednotný přístup k hardwaru: grafice (přes OpenGL, Vulkan, Metal), zvuku, klávesnici, myši a herním ovladačům.
- **Využití v praxi:**
  - Základ pro herní enginy, emulátory a multimediální aplikace.
  - V moderní verzi SDL3 došlo k výraznému vylepšení API (např. bezpečnější práce s ukazateli a lepší podpora moderních GPU API).

Praktická ukázka: SDL3

Inicializace knihovny a vytvoření prázdného okna

```c
#include <SDL3/SDL.h>
#include <stdio.h>

void create_window_example(void) {
    // Initialize the video subsystem
    if (SDL_Init(SDL_INIT_VIDEO) < 0) {
        printf("SDL_Init failed: %s\n", SDL_GetError());
        return;
    }

    // Create a window
    SDL_Window *window = SDL_CreateWindow("Advanced C: SDL3",
                                          800, 600,
                                          SDL_WINDOW_RESIZABLE);
    if (window == nullptr) {
        printf("Window creation failed: %s\n", SDL_GetError());
    } else {
        printf("Window successfully created!\n");
        SDL_DestroyWindow(window);
    }

    SDL_Quit();
}
```

libpng

Průmyslový standard pro čtení a zápis PNG obrázků

- **Co je to libpng:**
  - Oficiální referenční implementace pro formát PNG (Portable Network Graphics).
  - Extrémně rozšířená C knihovna, kterou pod kapotou používají webové prohlížeče, operační systémy i grafické editory.
- **Jak funguje (API):**
  - Knihovna je relativně nízkoúrovňová a využívá stavové struktury (`png_structp`, `png_infop`).
  - Umožňuje jemné nastavení transformací (např. automatický převod 16bitové hloubky na 8bitovou, odstranění alfa kanálu atd.).
  - Pro svou funkci vyžaduje kompresní knihovnu **zlib**.

Praktická ukázka: libpng

Příprava struktur pro čtení obrázku

```c
#include <png.h>
#include <stdio.h>

void read_png_init_example(FILE *fp) {
    // 1. Create and initialize the png_struct
    png_structp png_ptr = png_create_read_struct(PNG_LIBPNG_VER_STRING,
                                                 nullptr, nullptr, nullptr);
    if (!png_ptr) return;

    // 2. Allocate/initialize the memory for image information
    png_infop info_ptr = png_create_info_struct(png_ptr);
    if (!info_ptr) {
        png_destroy_read_struct(&png_ptr, nullptr, nullptr);
        return;
    }

    // 3. Set up error handling (libpng uses setjmp/longjmp)
    if (setjmp(png_jmpbuf(png_ptr))) {
        png_destroy_read_struct(&png_ptr, &info_ptr, nullptr);
        printf("Error reading PNG file!\n");
        return;
    }

    png_init_io(png_ptr, fp);
```

libcurl

Nástroj pro síťové přenosy a HTTP požadavky

- **Co je to libcurl:**
  - Multiplatformní knihovna pro přenos dat pomocí různých síťových protokolů (HTTP, HTTPS, FTP, SMTP a mnoho dalších).
  - De facto standard pro integraci webových API do C/C++ aplikací.
- **Klíčové vlastnosti:**
  - Podporuje synchronní (tzv. *easy* rozhraní) i asynchronní (tzv. *multi* rozhraní) síťovou komunikaci.
  - Automaticky řeší SSL/TLS certifikáty, cookies, proxy servery a přesměrování.
  - Architektura založená na callback funkcích (např. pro zápis přijatých dat do paměti).

Praktická ukázka: libcurl

Jednoduchý synchronní HTTP GET požadavek

```c
#include <curl/curl.h>
#include <stdio.h>

void fetch_url(const char *url) {
    CURL *curl = curl_easy_init();
    if (curl) {
        CURLcode res;

        // Configure the URL and tell curl to follow redirects
        curl_easy_setopt(curl, CURLOPT_URL, url);
        curl_easy_setopt(curl, CURLOPT_FOLLOWLOCATION, 1L);

        // Perform the request, res will get the return code
        res = curl_easy_perform(curl);

        // Check for errors
        if (res != CURLE_OK) {
            printf("curl_easy_perform() failed: %s\n",
                   curl_easy_strerror(res));
        }

        // Always cleanup
        curl_easy_cleanup(curl);
    }
```

SQLite

Plnohodnotná relační databáze v jednom souboru

- **Co je to SQLite:**
  - Knihovna napsaná v C implementující malý, rychlý, soběstačný a vysoce spolehlivý SQL databázový engine bez potřeby konfigurace.
  - Nejrozšířenější databázový stroj na světě (nachází se ve všech mobilních telefonech a většině moderních aplikací).
- **Integrace do C:**
  - Nevyžaduje běžící server (žádné démony, žádné procesy na pozadí). Čte a zapisuje přímo do běžných diskových souborů.
  - Může být integrován jako takzvaná *Amalgamation* (veškerý kód databáze spojený do jediného souboru `sqlite3.c`).

Praktická ukázka: SQLite

Otevření databáze a vytvoření tabulky

```c
#include <sqlite3.h>
#include <stdio.h>

void init_database(void) {
    sqlite3 *db;
    char *err_msg = nullptr;

    // Open a database file (or create it if it doesn't exist)
    int rc = sqlite3_open("test.db", &db);
    if (rc != SQLITE_OK) {
        printf("Cannot open database: %s\n", sqlite3_errmsg(db));
        sqlite3_close(db);
        return;
    }

    // Execute a simple SQL statement
    const char *sql = "CREATE TABLE IF NOT EXISTS Users(Id INT, Name TEXT);";
    rc = sqlite3_exec(db, sql, 0, 0, &err_msg);

    if (rc != SQLITE_OK) {
        printf("SQL error: %s\n", err_msg);
        sqlite3_free(err_msg);
    } else {
        printf("Table created successfully.\n");
```

Samostatný úkol: In-memory SQLite logger senzorů

Časová dotace: 15–20 minut

Zadání úkolu:
Využijte externí knihovnu **SQLite** k vytvoření rychlé databáze v operační paměti (`":memory:"`), uložení naměřených hodnot ze senzorů a jejich vyčtení pomocí callback funkce.

Požadavky na implementaci:

1. Otevřete databázi pomocí `sqlite3_open(":memory:", \&db)` (nevytváří soubor na disku, běží čistě v RAM).
2. Pomocí `sqlite3_exec` vytvořte tabulku `Sensors(Id INT, Temp REAL)` a vložte do ní alespoň 3 záznamy.
3. Napište vlastní C callback funkci a zavolejte `sqlite3_exec` s dotazem `"SELECT * FROM Sensors WHERE Temp > 25.0;"`, který vypíše vyfiltrované záznamy do konzole.

Finální projekt (Capstone)

Finální projekt: Průmyslový telemetrický uzel

Časová dotace: 120 minut | Architektura a požadavky (1/2)

Cíl projektu:
Navrhněte a implementujte kompletní, deterministický a plně testovatelný subsystém průmyslové telemetrické jednotky v jazyce **C23** bez použití dynamické haldy (`malloc`/`free`).

Fáze 1 – Příjem dat z přerušení a správa paměti (35 min):

1. **SPSC Ring Buffer & Atomics:** Implementujte bezzámkové FIFO (kapacita 2^N) s atomickým počítadlem přetečení (`<stdatomic.h>`), do kterého simulované přerušení (ISR) zapisuje surové vzorky.
2. **Memory Pool ($O(1)$):** Vytvořte statický bazén pevných bloků (`TelemetryNode`) pro alokaci zpracovávaných událostí v konstantním čase.

Fáze 2 – Polymorfní události a intruzivní fronta (30 min):

3. **Tagged Union:** Každý uzel v bazénu obsahuje typovaný C23 `enum : uint8_t` a anonymní `union` (pro teplotní vzorek, stav motoru nebo kritický alarm).
4. **Intruzivní seznam (`container_of`):** Alokované události řaďte do čekající fronty pomocí vnitřního uzlu `ListNode` a makra `CONTAINER_OF`.

Finální projekt: Průmyslový telemetrický uzel

Časová dotace: 120 minut | Architektura a požadavky (2/2)

Fáze 3 – Big-Endian serializace, DMA zarovnání a HAL (30 min):

5. **Bit-fields & Big-Endian:** Sbalte stavové příznaky pomocí bitových položek a serializujte událost byte-po-bytu ve formátu **Big-Endian** bez závislosti na paddingu.
6. **DMA & Cache Line:** Výstupní vysílací buffer zarovnejte na hranici 32B Cache Line pomocí `alignas(32)` a velikost zaokrouhlete makrem `ALIGN_UP_32`.
7. **HAL & Dependency Injection:** Komunikaci s vysílačem a stavovým registrem (`volatile`) oddělte pomocí struktury funkčních ukazatelů `RadioHalInterface` a ošetřete chyby pomocí `[[nodiscard]]` a `goto`.

Fáze 4 – Unit testování a regresní test (25 min):

8. **Unity Mock Test:** Vytvořte mock implementaci rozhraní `RadioHalInterface` a napište unit test ověřující správnou Big-Endian serializaci a uvolnění bloku zpět do Memory Poolu.
9. **Regresní test:** Napište test ověřující odolnost proti vyčerpání kapacity Memory Poolu a neplatné délce rámce.

Finální projekt: Architektonické schéma uzlu

Datový tok od hardwarového přerušení po DMA přenos

```
// Data flow pipeline implemented in the Capstone Project:
//
// [HW Sensor ISR]
//        | (1. Lock-free SPSC Ring Buffer + <stdatomic.h>)
//        v
// [Worker Task: O(1) Memory Pool Allocator]
//        | (2. Tagged Union Event + Intrusive List via CONTAINER_OF)
//        v
// [Big-Endian Serializer & Bit-field Packer]
//        | (3. alignas(32) DMA TX Buffer + [[nodiscard]] + goto cleanup)
//        v
// [Injected RadioHalInterface (Real volatile MMIO or Unity Mock)]
```

Budeme velmi rádi, pokud nám poskytnete hodnocení tohoto kurzu.



Hodnocení školení

Tato stránka slouží účastníkům k vyplnění zpětné vazby během probíhajícího školení.

Přístup je aktivní pouze po dobu konání kurzu a následující 3 dny.

Kód kurzu

C2

Jméno

Zadejte své jméno

Příjmení

Zadejte své příjmení

Vyplnit hodnocení

Vaše odpovědi jsou u nás v bezpečí a slouží výhradně ke zlepšení kvality našich školení.

ictPRO

Děkuji za pozornost!